

Machine Learning for Biology

QASIM HUSSAIN

We've essentially found a way to make sand think. It's miraculous.

Demis Hassabis

Nobel Laureate · Co-Founder and Chair, Google DeepMind · Chief Scientist, Alphabet ·
Founder and CEO, Isomorphic Labs

“A Framework for Frontier AI and the Dawning of a New Age”, 14 July 2026

CONTENTS

	Preface	4
	Prologue	6
PART I · LEARNING FROM LABELS		
1	Predicting antibiotic resistance from a genome Logistic regression	9
2	What does a cell show to the immune system? Decision tree	26
3	Is a gene a fact or a call? Random forest	46
4	What happens to the patients who are still alive? Gradient boosting	64
5	Gut bacteria and cardiovascular risk Logistic regression (support vector machine planned)	73
6	Neoantigen immunogenicity for cancer vaccine design Neural network	85
PART II · LEARNING WITHOUT LABELS		
7	Immune cell subtype discovery from single-cell RNA-seq k-means	95
	Epilogue	107

PREFACE

This book began as a series of daily posts. Between 24 July and 1 October 2026, I posted seventy of them on machine learning for biology, one a day, each written to stand on its own. Here, the first and the last are rewritten as the prologue and the epilogue, and the rest as seven chapters, with one model and one biological question to each.

Chapter 1 uses logistic regression to ask whether antibiotic resistance can be read straight from a bacterial genome. Chapter 2 uses a decision tree to ask which fragments of a protein, called peptides, a cell will show to the immune system. Chapter 3 uses a random forest to ask whether the disagreements between two annotation tools, programs that find the genes in a genome and name them, can be predicted from the DNA alone. The two read the same genomes and often name the same gene differently. Chapter 4 uses gradient boosting to rank cancer patients by their risk of recurrence or death when most of them have not had the event the study is waiting for. Chapter 5 sets out to use a support vector machine on gut bacteria and cardiovascular risk, finds that the public data cannot support its design, and answers two narrower questions of a public dataset with logistic regression instead. Chapter 6 uses a neural network to ask which peptides on a tumour cell a T cell will notice.

Those six are supervised. They learn from examples that someone has already labelled. Chapter 7 is not. It uses k-means on single-cell RNA sequencing (RNA-seq) data and has to find the groups of immune cells without ever being shown one.

The method stays the same throughout. The rules are written down before any model is fitted, and in most chapters so are the ways the analysis could go wrong. Every result is set against a baseline it has to beat, and the baselines are deliberately plain. They include guessing the commonest answer, a textbook rule about one position in a peptide, a model given only the stage a clinician already knows, and one given nothing but how tightly each peptide binds the molecule that displays it. Results that failed are reported beside results that held, and some chapters are mostly about the failures.

The prologue explains where this started, and the epilogue sets the seven models side by side at the end. I wrote the book for two kinds of reader: biologists who want to see what a model is doing, and people who build models and want to see where biological

data can mislead them. Terms from either side are explained where they first appear, many of them in short notes beside the text.

Every chapter has a public repository with its code, its data pipeline and the numbers it reports. The end of each chapter links to that repository, pinned to the commit the chapter describes, and the code listings in the text are quoted from that commit.

The chapters can be read in order, from logistic regression to k-means, or one at a time. Each opens with a summary of its question and its main results.

PROLOGUE

Cancer patients at Duke University were assigned to chemotherapy regimens chosen by a machine learning model. The model was wrong. Proving it took two statisticians more than 1,500 hours (Coombes, 2012), because the code and the processed data behind it were never released in full.

The method was published in *Nature Medicine* in 2006. It read the gene expression profile of a patient's tumour and predicted which drug that tumour would respond to. Oncology wants this badly, and three clinical trials were built on it. What went wrong is the reason this book exists. None of the errors found later was in the algorithm.

Keith Baggerly and Kevin Coombes, biostatisticians at MD Anderson, tried to reuse the method and could not reproduce it. So they reverse-engineered the analysis from the published results and the files the authors had posted, a practice they called forensic bioinformatics (Baggerly and Coombes, 2009). They found sample labels reversed between the responder and non-responder groups, and gene lists shifted by one position against their identifiers. Off-by-one errors and mislabelled columns: the kind of thing that never appears in a methods section. The trials were suspended.

The lesson generalises far beyond this case. In biology, most machine learning failures are not modelling failures. They are data failures wearing a model's clothes. An accuracy figure means nothing until you know the provenance of every label that produced it, and checking that requires understanding the laboratory assay, not the model's architecture.

WHY I AM WRITING THIS

When I began writing the daily posts this book is rewritten from, I had open-sourced around sixty computational biology analyses: metagenomic co-assemblies, pangenomes, single-nucleus atlases, cross-tissue transcriptome-wide association studies, spatial transcriptomics. Almost none of it was machine learning.

It was the half of the problem that Baggerly and Coombes were doing: provenance, labels, the orientation of a matrix. The other half, building models, was missing. Both halves are required, and most people I meet are missing one of them.

So I set out to spend the next year, or years, helping everyone close that gap. The plan was foundations first, then classical methods on omics data, then deep learning, sequence models, and generative and agentic systems. The code, the notebooks, the mistakes and the dead ends would all be open-sourced and public. This book covers the first part of that plan, the basics of machine learning in seven models from logistic regression to k-means, and it stops there on purpose.

This prologue is rewritten from the post for Day 1 of the series.

REFERENCES

- Baggerly, K. A. and Coombes, K. R. (2009). Deriving chemosensitivity from cell lines: Forensic bioinformatics and reproducible research in high-throughput biology. *The Annals of Applied Statistics*, 3(4), 1309-1334. <https://doi.org/10.1214/09-AOAS291>
- Coombes, K. R. (2012). The need for publicly verifiable and reproducible data and analyses. Slides for a talk at Research Integrity, Mohonk, 8 August 2012. <https://www.uab.edu/norc/images/conferences/documents/KCoombes-Mohonk-Aug-2012.pdf>

PART I

LEARNING FROM LABELS

PREDICTING ANTIBIOTIC RESISTANCE FROM A GENOME

Model: Logistic regression

If resistance is written in the genome, why can nobody simply read it off?

SUMMARY

Across 3,984 *Campylobacter* genomes with ciprofloxacin results measured on a plate, one question, whether *gyrA* carries a substitution at position 86, is right 99.67% of the time, against a floor of 79.32% for guessing susceptible, and a logistic regression given all 84 resistance genes does no better, at 99.60% under cross-validation. Most of the chapter is about what that number does not mean. The label hides the efflux pump; a regression on the 58 genes found in at least three isolates, fitted with families of related genomes kept together, pays a beta-lactamase the same weight as the gyrase mutation it always travels with; a rarity threshold of ten, in an early version of the pipeline, deleted a resistance mutation without warning; and one of the four changes at position 86 points the other way. Excluding it raises the score to 99.77%, a change made after looking and reported as such.

Campylobacter has 1.6 million bases of DNA. Change one of them, a single C to a T, and ciprofloxacin stops working. One letter out of 1.6 million, and that is not a figure of speech.

Fluoroquinolones, the family ciprofloxacin belongs to, kill bacteria by jamming the enzymes that untangle DNA during replication. *E. coli* has two of them, DNA gyrase and topoisomerase IV. Mutate one and the drug still has the other, which is why resistance in *E. coli* usually needs hits in both. *Campylobacter jejuni* has no topoisomerase IV. Repeated attempts to find *parC* have failed, and the genome does not carry it. One target, and no backup.

A single base change at position 257 of *gyrA*, turning threonine 86 into isoleucine, raises the ciprofloxacin MIC 128-fold. The two numbers name one change: 257 counts bases in the gene, and 86 counts amino acids in the protein it encodes, which forms part of DNA gyrase.

Minimum inhibitory concentration (MIC). The lowest concentration of a drug that stops the organism growing.

Appearing is one thing. Lasting is another. Most human *Campylobacter* infection comes from poultry, so what matters is whether the mutation survives inside a chicken, with no drug anywhere near it. It does. Resistant and susceptible strains colonised chickens equally well with no antibiotic present. Nothing pushes the mutation back out, and once it appears, it stays.

So the answer is in the genome, in a known gene at a known position. The clinic still does it the slow way: grow the organism, expose it to the drug, wait. *Campylobacter* is microaerophilic and slow, so that takes days, and the patient is treated on a guess in the meantime. Sequencing takes a day. Read position 257 and you are done.

Except that you are not. Take two isolates, each a culture of the bacterium grown from one sample, that both carry the change at 257. One dies at a concentration of ciprofloxacin that the other survives. *Campylobacter* has a pump, CmeABC, that pushes the drug back out of the cell before it ever reaches the gyrase. Disable the pump and the resistance collapses, with the mutation still intact.

Position 257 is not a switch. It is a contribution, and how much it contributes depends on what else the genome is carrying. That is the question this chapter is built on. If resistance is written in the sequence, why can nobody simply read it off? Half of the answer is in the biology. The other half, as it turned out, is in the machine learning.

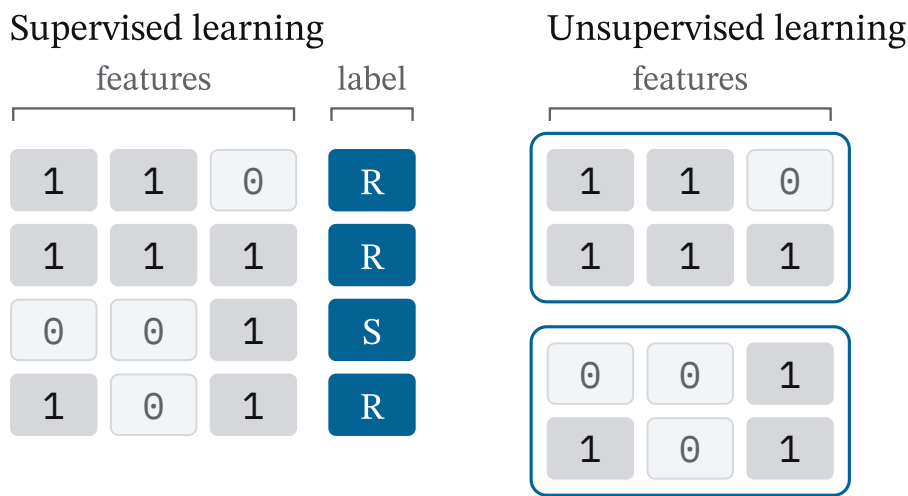
1.1 FEATURES, LABELS AND A FLOOR

The models in this chapter run off a table, with one row per isolate and one column per resistance gene, holding a 1 where the gene is present and a 0 where it is not. A known resistance mutation, such as threonine to isoleucine at position 86 of *gyrA*, gets a column of its own and counts among the resistance genes. What a model looks at are its features, and here the features are those columns. What it predicts is its label: for each isolate, susceptible or resistant, as the laboratory measured it. Every isolate arrives with its answer attached, and the model has to learn from those answers how to give the right one for a genome it has not seen. That is supervised learning. Figure 1.1 sets it beside the other possibility, unsupervised learning, where no answers are given and the job is to find the groups. Nothing in this chapter asks for that.

The first model is a rule, with nothing fitted to the data. It asks one question of each genome. For ciprofloxacin, the biology supplies the question: is there a substitution at position 86 of *gyrA*? It asks nothing else: no pump, no context, no second gene.

The second model is a logistic regression. I gave it all 84 resistance genes and let it weight them freely. For each genome, the model adds the weights of the genes it carries to a starting value shared by every genome, turns the total into a probability of resistance, and calls the isolate resistant when that probability is above one half. A gene with a large weight is one the model leans on, and a gene with a weight near zero is one it barely looks at.

Before either model is believed, it needs something to beat. Most of the isolates in this chapter are susceptible to ciprofloxacin, so a model that reads no DNA at all and guesses susceptible every time is right 79.32% of the time. That is the floor. Find the floor before you believe any number, including mine.



Answers given:
learn to predict them

No answers:
find the groups

Figure 1.1 One table drawn twice, a row for each isolate and a column for each resistance gene, with 1 where the gene is present and 0 where it is not. In supervised learning each row arrives with its label, resistant (R) or susceptible (S), and the model learns to predict it; in unsupervised learning there is no label, and the job is to find groups among the rows. The ones and zeros are illustrative.

1.2 THE RULES

Both rules were fixed before any modelling, and each was changed once after I had looked at the data. Each changed version sits beside its original, with the reason for the change, and the sections that follow explain each one.

Ciprofloxacin, fixed before any modelling.

Call an isolate resistant if *gyrA* carries any substitution at position 86.

Ciprofloxacin, changed after looking.

The same rule without alanine at position 86, on the strength of a separate assay with nalidixic acid and a published result (Jesse and colleagues, 2006).

Tetracycline, fixed before any modelling.

Call an isolate resistant if the genome carries any *tet* gene.

Tetracycline, changed after looking, kept as a hypothesis.

The same rule, with a *tet* gene counted as absent where the annotation reports only partial copies or copies broken by an internal stop or a frameshift, because the mechanism says a truncated protein cannot work. A copy reported as partial because it ran off the end of a contig still counts as present.

1.3 ONE POSITION AGAINST EIGHTY-FOUR GENES

I took 3,984 *Campylobacter* genomes from NCBI. Every one had a ciprofloxacin result measured on a plate. The one question about position 86 was right for all but thirteen of them, which is 99.67%. That beats the floor by twenty points. The logistic regression, with all 84 genes to draw on, did not beat the one question. Its score is cross-validated, averaged over repeated divisions of the data and each time measured only on isolates held back from its training. It was right 99.60% of the time, and 99.57% when whole families of related genomes were held back together. The rule has nothing to fit, so it scores 99.67% either way.

The pump is real, and I modelled none of it. A single question was still right 3,971 times. The reason has nothing to do with the bacterium. It is about what the laboratory measured.

What the laboratory measured was a number. The organism goes into a row of wells, each holding twice as much ciprofloxacin as the one before it, and the lowest concentration that stops it growing is the result. That number is continuous. Some isolates stop at a low dose, and some need far more. Then somebody draws a line across it, susceptible below, resistant above.

A good pump buys the organism another tube or two. If the isolate stays on the same side of the line, the pump did something and the label recorded nothing. So I did not solve the problem I opened with, which was how much a change contributes. I answered a newer one: which side of the line an isolate falls on.

That lesson arrives before any algorithm does. The label is a decision somebody made, usually before you arrived, often for convenience. Choose the label and you have fixed the ceiling. Every model after that, however clever, works underneath it.

The weights showed the next problem. I read them from the same kind of model fitted to a narrower table, the 58 genes found in at least three isolates, using the training isolates of a split that kept families of related genomes together. The chapter comes back to both choices. That model gave the isoleucine change at position 86 a weight of 8.845. It gave a real gyrase mutation, the valine change at the same position, 2.364. It also gave a beta-lactamase 2.364: the same number, to three decimals, for a gene that does nothing to fluoroquinolones.

1.4 A GENE THAT CANNOT TOUCH THE DRUG

A beta-lactamase cuts open a beta-lactam ring. Penicillin has one, and so does amoxicillin. Ciprofloxacin has none, so there is nothing in it for the enzyme to cut.

The gene is *blaOXA-493*. Five isolates carry it, and all five are resistant to ciprofloxacin, so the model paid it. Now look at those five again. Every one also carries a gyrase mutation, the valine change at position 86, with no exceptions.

Put yourself where the model sits. You see two columns. Wherever the gene is present, the mutation is too, and the isolate is resistant. Which one is responsible? Nothing in the table can tell you. Trained with families kept together, the 58-gene model split the credit, 2.364 each, because it had no other option. That is confounding, the oldest problem in statistics wearing a genomic coat (Figure 1.2).

Isoleucine at *gyrA* position 86



Valine at *gyrA* position 86



blaOXA-493, a beta-lactamase



blaOXA-493 never appears without the valine change



Figure 1.2 Weights given to three features by the logistic regression on the 58 genes found in at least three isolates, trained with families of related genomes kept together: 8.845 for isoleucine at *gyrA* position 86, and 2.364 each for valine at the same position and for the beta-lactamase gene *blaOXA-493*, which never appears without the valine change.

The part that should worry you comes next. Suppose a sixth isolate turns up carrying *blaOXA-493* and no gyrase mutation. The model counts the gene towards resistance and raises the probability it gives that isolate, for a reason that has nothing to do with the drug. There is no such isolate among these 3,984. The mistake is already in the model, and nothing in the data will show it to you.

The model did not learn what the gene does. It learned what the gene travels with. Why the two travel together I cannot tell from five isolates. Shared ancestry, perhaps, or nothing at all. The model cannot tell either, because it only ever saw them arrive as a pair.

This is the limit of the whole method. A model sees co-occurrence, and that is all it ever sees. From inside a table, a cause and its companion look identical. What separates them is knowing what the gene does, and that comes from biology, from somebody who knows that ciprofloxacin has no beta-lactam ring.

1.5 HIDING WHOLE FAMILIES

If the model is reading company rather than mechanism, there is a way to catch it: hide whole families of isolates from the model, then see whether it still works.

To see why that helps, start with how a model gets tested at all. You hold data back. Train on three quarters of the isolates, then score the model on the quarter it has never seen. If it performs well on that quarter, it has learned something. That only works if

the held-back quarter is genuinely new to it.

These isolates are not strangers to one another. NCBI sorts them into families called SNP clusters, a SNP being a difference at a single base. Two genomes in the same cluster differ by a handful of bases out of 1.6 million: the same organism sampled twice, often from two patients in one outbreak. Of my 3,984 isolates, 3,520 fall into 1,128 clusters, and the other 464 have no cluster assignment. Split them at random and 344 of those clusters end up with isolates on both sides. The model learns from one genome and is then tested on its near-identical twin. It scores well and has learned nothing, because it is recognising a genome it has already seen.

Data leakage. Information reaching the test set through a back door, here through near-identical genomes on both sides of the split.

So I split again, keeping every cluster whole, with no family on both sides. The 344 became zero (Figure 1.3). Listing 1.1 shows the split.

Listing 1.1 The grouped split, which keeps every SNP cluster on one side of the divide, from `scripts/07_split_data.py`.

```
# Grouped split. Isolates with no cluster assignment are treated as
# their own singleton group, which is the conservative reading: we
# cannot show they are related to anything, so we do not assume it.
groups = y["snp_cluster"].fillna(
    pd.Series([f"__unclustered_{i}" for i in idx], index=y.index)
)

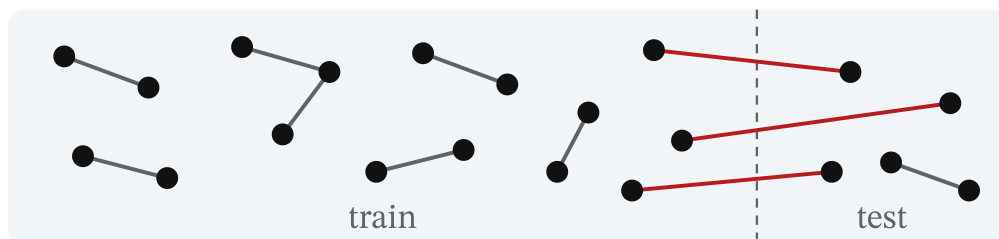
gss = GroupShuffleSplit(n_splits=1, test_size=TEST_FRACTION,
random_state=SEED)
train_g, test_g = next(gss.split(idx, y["resistant"], groups))
y["split_grouped"] = "train"
y.loc[test_g, "split_grouped"] = "test"
```

A single division of the data can be accidentally kind, so I repeated the comparison twenty-five times, dividing the data differently each time. Averaged over those twenty-five, the one question about position 86, asked of the 58-gene table, was right for 0.9965 of the held-back isolates with families allowed on both sides and for 0.9965 with families kept whole, and the 58-gene logistic regression was right for 0.9961 and 0.9960. The rule's number did not move, and the regression's barely did.

That is good news. If the regression had been reading family resemblance, it would have failed on families it had never seen. It did not fail. It predicted resistance in families it had never encountered, and that is what learning a mechanism looks like.

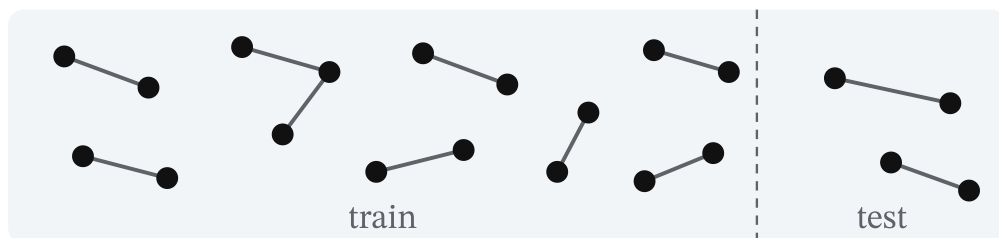
The beta-lactamase is still in there, though. Three of the eight valine isolates carry no *blaOXA-493*. The random split left valine isolates without the gene on the training side, so the 58-gene model could give the valine change more of the credit: 3.689, against 1.469 for the beta-lactamase. The grouped split, which gave the weights quoted earlier, held all three back for testing, so the model never saw the two apart and paid them 2.364 each. Splitting by family did not remove the gene's weight, and it could not. Every isolate that carries *blaOXA-493* also carries the *gyrA* mutation, so however the data is divided, the two are still together on whichever side they fall.

Split at random



344 of 1,128 families on both sides

Split by family



0 families on both sides

Figure 1.3 A sketch of isolates, each a dot, with lines joining isolates of the same family, a SNP cluster of near-identical genomes. Split at random into training and test sets, 344 of the 1,128 families have isolates on both sides of the divide; split by family, none do.

So there are two problems here, and they are different problems. Leakage is about how you divide the data, and dividing it better fixes it. Confounding is about what is in the data, and no division fixes it at all.

1.6 THE GENE WAS THERE; THE PROTEIN WAS NOT

A different drug exposed a different weakness. Tetracycline sits down on the ribosome and blocks the seat where each new amino acid has to arrive. Protein synthesis stops, and the cell cannot build anything. *Campylobacter*'s answer is a gene called *tet(O)*. It makes a protein that binds the ribosome and dislodges the drug.

So the rule is easy. If the genome carries a *tet* gene, call the isolate resistant. In these genomes, that means *tet(O)*, except in five isolates that carry one of two mosaic genes instead, *tet(O/M/O)* in four and *tet(O/W/32/O)* in one, all five of them resistant. Across 3,983 isolates, every one with a tetracycline result measured on a plate, that rule is right 99.32% of the time. A rule that asked about *tet(O)* alone would miss those five and score 99.20%. But fourteen isolates carry *tet(O)* and die of tetracycline.

Carrying a gene is one thing. Making a working protein is another. Part of the explanation lay in how the gene had been written down. AMRFinderPlus is the software that reads a genome and reports which resistance genes are in it, and that report is the annotation. It works by comparing the sequence against a reference copy of each known gene, and it does not answer yes or no. When the match covers the whole reference gene, it writes *tet(O)*. When the match covers only part of it, it writes *tet(O)=PARTIAL*. My rule had been reading both as present.

Of these isolates, 1,890 carry a full-length copy, and 99.6% of them survive tetracycline. Four carry a fragment and nothing else, with no full copy anywhere in the genome. All four died. The protein, Tet(O), has to reach the ribosome and take hold of it. Part of a Tet(O) reaches nothing.

There is a second kind of partial. Fifteen isolates carry *tet(O)=PARTIAL_END_OF_CONTIG*, and 93% of those are resistant. That flag means something entirely different. Sequencing does not hand you a genome in one piece. It hands you short reads, which software assembles into longer fragments called contigs, and in these isolates the gene ran off the edge of one. The sequence stops because the assembly stops, not because anything is missing from the bacterium. One word, partial, was covering two situations. One concerns the organism, and the other concerns how its genome was pieced together.

Separate them, and 99.32% becomes 99.45%. That is five isolates out of 3,983: the four that carry only a fragment, and one whose only copies are a fragment and a copy broken by an internal stop or a frameshift. I ran this comparison twenty-five times as well. A rule has nothing to fit, so the repeats cannot raise or lower its average. What they show is how far the difference between the two rules moves from one division of

the data to the next, and whether a gain of five isolates is larger than that. It is about the same size. The gain is 1.1 times that spread, and anything below about 1 cannot be told apart from noise. The mechanism says a truncated protein cannot work. The data agrees, quietly, and cannot prove it, so the change goes into the repository as a hypothesis. Nine of the fourteen are still unexplained: an intact gene, susceptible on the plate, and no reason I can give.

The lesson underneath does not depend on whether that difference is significant. My table already kept `tet(0)` and `tet(0)=PARTIAL` in separate columns. The rule merged them. It counted every *tet* entry as present, so it read a fragment of *tet(O)* the same as a whole copy. The annotation had been telling me more than that, and I threw it away before the rule ever ran.

1.7 A THRESHOLD CHOSEN IN SECONDS

Back to ciprofloxacin, and to the table. Before any model uses it, you have to decide which columns to keep. Here is why you would drop any. Suppose a gene turns up in two isolates out of 3,984, and both happen to be resistant. In the table, that column looks perfect. It is also worthless. Two isolates tell you nothing about a gene, and the model cannot know that.

So the standard move is to delete rare features. Pick a minimum count and cut everything below it. In an early version of the pipeline, I picked ten. Anything found in fewer than ten isolates was gone before the model ran.

Now look at what that removed. The change this chapter opened with turns threonine at position 86 of *gyrA* into isoleucine. There is a second change at the same position, threonine to valine. Eight isolates in this cohort carry it, and every one of them is resistant. Those eight sit in seven different SNP clusters, counting an isolate with no cluster as a cluster of its own, as the split does. So they are seven independent observations rather than one outbreak counted repeatedly. Eight is fewer than ten.

With the threshold at ten, the rule asking about position 86 could only find the isoleucine version, and it was right 99.55% of the time. Lower the threshold to five, so that the valine version survives the cut, and the same rule is right 99.75% of the time. The gap is eight isolates, exactly the eight carrying the change I had deleted (Figure 1.4). Both thresholds also cut the two rarest changes at that position, which come later in the chapter, so neither figure is the 99.67% that the rule scores when it counts every change.

Change at <i>gyrA</i> position 86			
	810 isolates	8 isolates	Rule scores
Threshold 10	isoleucine	valine	99.55%
Threshold 5	isoleucine	valine	99.75%

Figure 1.4 A rarity threshold keeps a gene column only if at least that many isolates carry it. At ten, the setting of an early version of the pipeline, it cuts the column for the valine change at *gyrA* position 86, carried by eight isolates, and the rule scores 99.55%; at five, the column stays, and the rule scores 99.75%. The isoleucine change, in 810 isolates, is kept at both.

Nothing warned me. There was no error, no missing file and no collapse in accuracy. Both numbers sit above 99%, and both look like a model that works. Ten was a number I chose in a few seconds because it sounded reasonable, and I did not test it. It removed the best-evidenced resistance mutation in the dataset after the common one.

I have tested it since. The pipeline now scores the rule and the logistic regression with the threshold at 1, 3, 5, 10 and 20, and at no setting does the regression beat the rule. The default is three, the smallest count that removes a two-isolate column like the one above. That is why the valine change, in eight isolates, and *blaOXA-493*, in five, both carry weights in the model I read them from. Three is a choice too, and it still cuts the rarest change at position 86, which a single isolate carries.

So the model I trained was the algorithm plus every decision I made before it started: which columns to keep, what counts as one sample, what the label is allowed to mean. None of those show up in an accuracy figure. All of them decide what that figure is measuring.

1.8 FOUR CHANGES AT ONE POSITION

Position 86 is the amino acid in DNA gyrase that sits exactly where ciprofloxacin has to bind. Change it and the drug loses its grip. Four different changes turn up there in these 3,984 genomes. Threonine normally sits at 86. In 810 isolates it has become isoleucine, in eight valine and in one lysine. That is 819 isolates, and 817 of them survive ciprofloxacin.

The fourth change is alanine. Four isolates carry it, and none of the four is resistant.

The explanation came from a second drug, nalidixic acid, the original quinolone, in clinical use since the 1960s. Ciprofloxacin came later. It is a fluoroquinolone, a far more potent version of the same idea, aimed at the same enzyme. The NCBI release that supplied these genomes also holds nalidixic acid results for 3,816 sequenced isolates, the four alanine isolates among them. All four are resistant to nalidixic acid. Not one of them is resistant to ciprofloxacin.

The other three substitutions defeat both drugs. Against nalidixic acid, counting only isolates with a result for it, the count for isoleucine is 772 resistant out of 775, for valine eight out of eight and for lysine one out of one. Alanine defeats only the old drug (Figure 1.5).

Look at what each change does to the enzyme. Isoleucine and valine are bulky. Lysine is long and carries a charge. Alanine is the smallest substitution available, barely a change at all. It is enough to shake off nalidixic acid and not enough to shake off ciprofloxacin.

None of this is new. Jesse and colleagues reported exactly this in 2006, in isolates that resisted high concentrations of nalidixic acid and were killed by low concentrations of ciprofloxacin. My four isolates reproduced a twenty-year-old result without being asked to.

Threonine 86 becomes	Resistant, of those with a result	
	nalidixic acid	ciprofloxacin
isoleucine	772 of 775	808 of 810
valine	8 of 8	8 of 8
lysine	1 of 1	1 of 1
alanine	4 of 4	0 of 4

Figure 1.5 The four substitutions at *gyrA* position 86, with the isolates resistant to each drug out of those that carry the substitution and have a result for that drug. Isoleucine, valine and lysine defeat both drugs; the four alanine isolates all resist nalidixic acid, and none of them resists ciprofloxacin.

Now the part that concerns the model. My feature was any substitution at position 86 of *gyrA*. I chose it from the mechanism, and the opening of this chapter argued for it. That feature covers four substitutions, and one of them points the other way. Drop alanine from the rule and it goes from 99.67% correct to 99.77%.

That is four isolates. Three of the four sit in the same SNP cluster, so it is closer to two independent observations than four. The improvement is a fraction of the variation between cross-validation folds, the repeated divisions of the data into training and test sets. The biology is published. The statistics here are silent. Both of those are true at once.

The discussion of the label, earlier in the chapter, was about the resolution of the answer, where susceptible or resistant is a line drawn across a continuous measurement. This is about the resolution of the question. How finely you define a feature is a choice, and taking it from the mechanism does not protect you. The mechanism at position 86 turned out to be four mechanisms.

1.9 CHANGING THE RULE AFTER LOOKING

I found the four alanine isolates by reading through the data, and then changed the rule to account for them. That sequence is how results go wrong.

The number went up because I moved the goalposts after watching where the ball landed. Do that often enough and you can improve any model to any level you like. Every dataset contains patterns that mean nothing, and a rule shaped around them will not survive contact with the next dataset.

So the safe answer would be to throw the improvement away and keep the original number. That answer is wrong too. Refusing to learn anything from data you have already seen looks like rigour. It is superstition. The real test is where the justification for a change comes from.

I have two such reasons, both set out above. The nalidixic acid result came from a different experiment on the same four bacteria, not from the accuracy I was trying to raise. In the 2006 paper, Jesse and colleagues found the same thing in bacteria I have never seen. Neither of those could have come out of tuning.

So both numbers go into the repository with their labels attached, one rule fixed before any modelling and the other changed after looking (Figure 1.6). Nobody has to guess which is which.

Fixed before any modelling

Any change at *gyrA* position 86

99.67%



Found in the data: four alanine isolates, none resistant

Changed after looking

The same rule without alanine

99.77%

On the strength of a nalidixic acid assay
and a published result (Jesse and colleagues, 2006)

Figure 1.6 The two versions of the ciprofloxacin rule, each with its score and labelled with when it was set. Any change at *gyrA* position 86 was fixed before any modelling and scores 99.67%. The same rule without alanine was set after the four alanine isolates had been found in the data, on the strength of a nalidixic acid assay and a published result, and scores 99.77%.

Looking at your data is not the sin. Pretending you did not is.

1.10 THE LIST BEHIND THE VERDICT

Two genes in this dataset look identical from the numbers. The first appears in five isolates, all five resistant to ciprofloxacin, in four separate SNP clusters, and never once without a mutation at position 86 of *gyrA*, the change that actually causes the resistance. The second appears in six isolates, all six resistant, in five separate clusters, and never once without a mutation at that position either. One of them has nothing to do with ciprofloxacin at all. The other pumps ciprofloxacin out of the cell. Nothing in the table tells you which is which.

The first gene is *blaOXA-493*, the beta-lactamase from earlier in the chapter.

So I wrote code to find the other passenger genes, the ones that travel with the cause and do nothing themselves. The code starts from the rule this chapter has been testing, a change at position 86 of *gyrA*. It then looks for genes that appear only in isolates that rule already calls resistant, and that have no known way of acting on the drug. The first half of that is arithmetic. The second half is a list of gene names I typed into the script: *gyrA*, *parC*, *tet*, *23S*, *erm*. Five entries, covering the mechanisms I had in mind that afternoon (Listing 1.2).

Listing 1.2 The hand-typed list of mechanisms that the passenger search checks against, from `scripts/10_threshold_sweep.py`.

```
# Prefixes with an accepted mechanistic link to some drug in this chapter.
# Anything outside this set that tracks the rule perfectly is a passenger.
MECHANISTIC = ("gyrA_", "parC_", "tet(", "23S_", "erm(")
```

The second gene is *cmeB*. It is part of CmeABC, the pump *Campylobacter* uses to throw fluoroquinolones back out of the cell before they reach their target. The annotation reported *cmeB* in only six of the 3,984 genomes, the six counted above. *cmeB* was not on my list. So the code did exactly what it was told, and accused the pump (Figure 1.7).

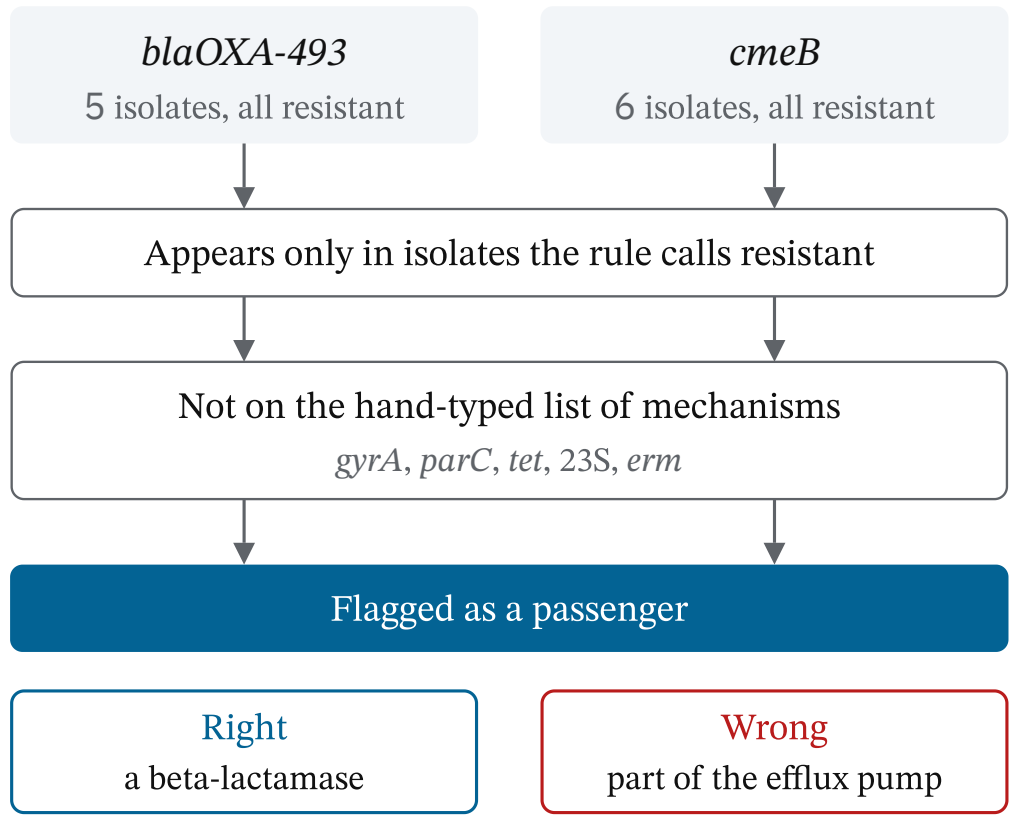


Figure 1.7 The passenger search applied to *blaOXA-493*, in five isolates, and *cmeB*, in six, all of them resistant. Both genes appear only in isolates the rule calls resistant, and neither is on the hand-typed list of mechanisms, so the code flags both as passengers: rightly for the beta-lactamase, and wrongly for *cmeB*, part of the efflux pump.

Look at the two genes again. Statistically they are the same gene. What separates them is knowing what the protein does, and here that knowledge was in a list. Mine had five entries and was missing one.

Being more careful next time would not fix this. Every method that sorts real causes from things that merely travel beside them uses knowledge from outside the dataset: a pathway database, a curated catalogue, a prior, somebody’s judgement. That knowledge is part of the method, and its gaps do not announce themselves.

1.11 WHAT SURVIVES

I set out to ask whether a genome can tell you what a laboratory plate would. For ciprofloxacin it can. One feature is right 99.67% of the time across 3,984 isolates, and eighty-four features do no better, at 99.60% under cross-validation. That part took an afternoon. The rest of the work went on finding out what the number does not mean (Figure 1.8).

The label made the problem easy, because the efflux pump moves an isolate a tube or two along the measurement without carrying it across the line, and so never reaches the answer. The regression on the 58 genes found in at least three isolates paid a gene that cannot touch the drug, because the gene never appears without a real gyrase mutation. Hiding whole families did not fix it. With families kept together, the regression paid the two the same. A rarity threshold I chose in seconds, in an early version of the pipeline, deleted a resistance mutation, and no accuracy figure noticed. The feature I built from the mechanism turned out to be four mechanisms. The code I wrote to find passenger genes named the efflux pump, because the list it checks against was mine, and incomplete.

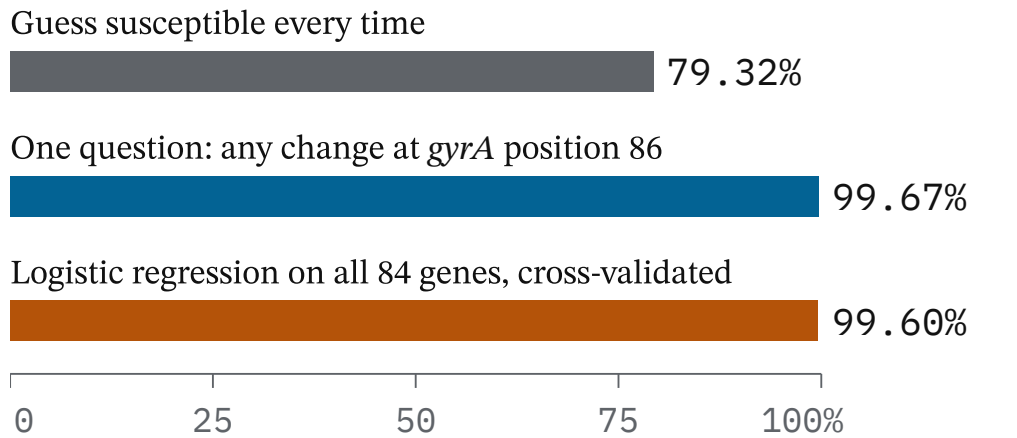


Figure 1.8 Three scores on the 3,984 isolates, drawn from zero: guessing susceptible every time, 79.32%; the one question about *gyrA* position 86, 99.67%; and the logistic regression on all 84 genes, 99.60% under cross-validation.

What survives all of that is narrow, and worth stating carefully. In *Campylobacter*, ciprofloxacin resistance is explained well enough by one position in one gene that nothing I built improved on it. That is a claim about this organism and this drug, and not about resistance prediction in general.

Everything is in the repository: thirteen scripts, one pinned NCBI release, the metrics file behind every result in this chapter, and a limitations section naming what the data could not establish. Nine tetracycline isolates in there carry an intact resistance gene and die anyway. I do not know why. Chapter 2 needs a harder problem.

This chapter is rewritten from the posts for Days 2 to 11 of the series.

REFERENCES

Jesse, T. W., Englen, M. D., Pittenger-Alley, L. G. and Fedorka-Cray, P. J. (2006). Two distinct mutations in *gyrA* lead to ciprofloxacin and nalidixic acid resistance in *Campylobacter coli* and *Campylobacter jejuni* isolated from chickens and beef cattle. *Journal of Applied Microbiology*, 100(4), 682-688. <https://doi.org/10.1111/j.1365-2672.2005.02796.x>

RUN IT

Repository github.com/Qasim-Hussain-Code/campylobacter_amr_phenotype_prediction
Commit [0e2b3df4c587](https://github.com/Qasim-Hussain-Code/campylobacter_amr_phenotype_prediction/commit/0e2b3df4c587)

CITE THIS CHAPTER

Hussain, Q. (2026). Predicting antibiotic resistance from a genome. In *Machine Learning for Biology*, chapter 1.

WHAT DOES A CELL SHOW TO THE IMMUNE SYSTEM?

Model: Decision tree

If the groove grips only two positions out of nine, why is one rule about position two not enough?

SUMMARY

One MHC molecule, HLA-A*02:01, and 11,723 laboratory measurements of nine-amino-acid peptides, of which 4,958 bound below 500 nanomolar. A decision tree given 27 columns of hydrophobicity, molecular weight and charge reached F1 0.7399 on proteins it had never seen, against 0.7153 for the same tree on 180 one-hot columns and 0.6667 for the textbook rule of leucine or methionine at position two. The tree recovered the two anchor positions published in 1991 without being told they existed. The chapter ends on an allele filter that matched a substring and let in 302 measurements from macaque, horse, cattle, chicken and eleven engineered point mutants of the human molecule.

There is a reason Chapter 1 stopped at logistic regression.

It was already enough. Eighty-four features did no better than one, and adding a more powerful model to a solved problem teaches nothing. So this chapter keeps the task, predicting a category, and changes the problem underneath it. The data is harder, and this time the differences between the models are real. Four models, one to a chapter: decision trees and the rules they learn, random forests and what is lost when a model can no longer be read, gradient boosting and a label that is two numbers together, and support vector machines and the geometry of a decision boundary (Figure 2.1). This chapter is the decision tree.

Right now, every cell in your body is holding up samples of what is inside it. Proteins get chopped into short fragments. Some of those fragments are carried to the cell surface and held there, where passing immune cells inspect them. If a fragment looks foreign, the cell is killed. That is how your immune system finds a virus hiding inside a cell without ever going in. It reads the samples on the outside.

The molecule doing the holding is called Major Histocompatibility Complex class I, or MHC class I. It has a groove. A fragment nine amino acids long sits in that groove like a key in a lock. Two of those nine do most of the gripping, the second and the ninth. Get both right and the fragment is displayed. Get one wrong and it falls out, the immune system never sees it, and a virus with the wrong fragments is invisible.

So here is the chapter. Give the model nine amino acids. Ask whether they bind.

One MHC molecule, HLA-A*02:01, the most studied variant in the world. The first download gave 15,597 laboratory measurements and 9,142 distinct fragments, and 53.7% of those measurements were binders, which is almost exactly a coin flip. In Chapter 1, guessing susceptible every time was right 79.32% of the time before I read a single base of DNA. This time, guessing gets me nothing. Those counts are the download as it arrived, before I checked which unit each measurement was in, and the corrected cohort, smaller and with a lower share of binders, comes later in this chapter.

One more thing, and it should feel familiar from Chapter 1. Binding strength is a number. Somebody drew a line across it and called everything on one side a binder. The database that supplies this data says, in its own documentation, that the line should probably sit somewhere different for every MHC molecule.

Nanomolar. The unit binding affinity is measured in. A smaller number means a tighter grip, so a measurement below 500 nanomolar counts as a binder.

Decision trees
the rules they learn

This chapter

Random forests
what is lost when a model can no longer be read

Gradient boosting
a label that is two numbers together

Support vector machines
the geometry of a decision boundary

Figure 2.1 The four models the chapters take in turn, one to a chapter, each with what it is there to show. This chapter is the first of them, the decision tree.

2.1 WHY A TREE

Now look at the shape of the groove's rule.

Chapter 1 was one position. One change at *gyrA* 86, and ciprofloxacin stops working. One question, one answer.

This needs two positions to agree at once. And what counts as agreeable at position nine depends on which MHC molecule you have, and people carry different ones. That is a conjunction, not a threshold.

Which is exactly what a decision tree is made of. It asks one question, then asks a different question depending on the answer, then another. Follow the path and you have a chain of conditions. The groove enforces a chain of conditions. The tree is built from them (Figure 2.2).

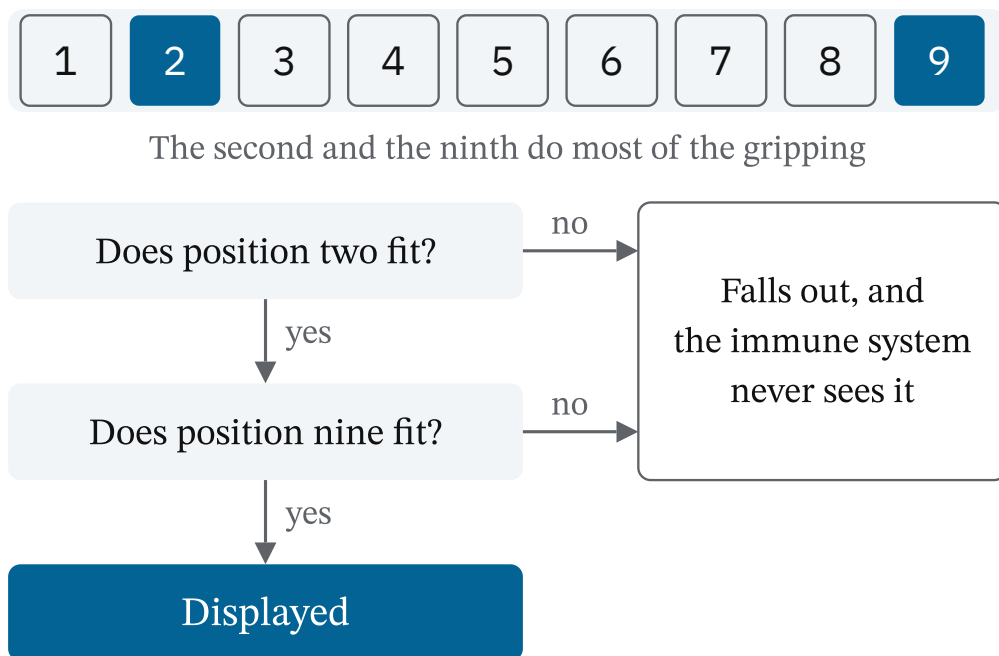


Figure 2.2 The groove holds nine positions and grips at two of them, the second and the ninth. Both have to fit, so the rule is a chain of two questions, which is what a decision tree is made of.

2.2 NINE LETTERS AS A TABLE

GILGFVFTL. Nine letters, positions 58 to 66 of the influenza A matrix protein. If you carry HLA-A*02:01, your cells display this fragment when you have flu, and it is the most studied T-cell target we have. Position two is isoleucine and position nine is leucine. Both sit in their pockets.

Now hand those nine letters to a model. It sees nine characters. No chemistry, no shape, no charge. Only characters.

Machine learning models take numbers, and a peptide sequence is text. Something has to convert one into the other, and that something is a choice.

The obvious choice is to number the alphabet. Alanine becomes 1, cysteine 2, and so on to tyrosine at 20. Nine columns. Done.

It is also wrong. A decision tree splits by asking whether a number falls below some value. Ask whether the amino acid is below ten and you have grouped alanine, aspartate, phenylalanine and lysine. They share nothing except the alphabet.

The honest version is called one-hot encoding, one column for every amino acid at every position. Nine positions and twenty amino acids give 180 columns, each holding a zero or a one. Every column is one yes-or-no question. Is position two a leucine. Is position three a valine. That is the question the groove asks.

The cost is that similarity disappears. Leucine and isoleucine are the same atoms arranged differently. Across 180 columns they are strangers, and the model has to learn each one alone.

The third option is to describe each amino acid instead of naming it. Hydrophobicity, molecular weight and charge become the columns, three numbers for every position and twenty-seven in all. Molecular weight stands in for the size of the residue, and similar residues then get similar numbers. That is fewer columns and more biology, but it is a bet placed before the model has seen anything.

All three go in the repository (Figure 2.3), and I said I would write down which one I expected to win before any of them had a score, because a choice made after seeing the scores is not really a choice. I never wrote it. The pipeline did not run for several days, and when it did, there was nothing on record saying which encoding I had backed. Listing 2.1 shows the three encodings.

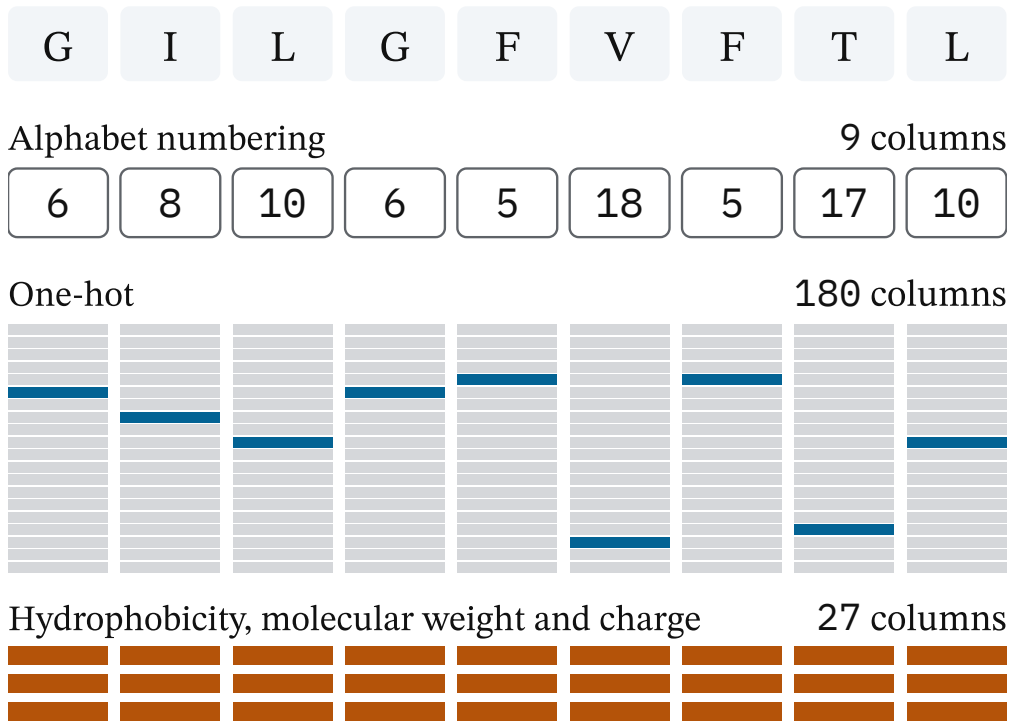


Figure 2.3 One peptide written three ways: nine columns of alphabet numbering, alanine 1 to tyrosine 20; 180 columns of one-hot, where one cell in twenty is set at each position; and 27 columns of hydrophobicity, molecular weight and charge.

Listing 2.1 The three encodings, from `pipeline.py`, where KD, MW and CH are lookup tables for hydrophobicity, residue molecular weight and charge.

```
def enc_ordinal(sequences):
    return np.array([[ORD.get(aa, 0) for aa in seq] for seq in sequences],
dtype=float)

def enc_onehot(sequences):
    rows = []
    for seq in sequences:
        row = []
        for aa in seq:
            row += [1 if aa == a else 0 for a in AA20]
        rows.append(row)
    return np.array(rows, dtype=float)

def enc_physico(sequences):
    rows = []
    for seq in sequences:
        row = []
        for aa in seq:
            row += [KD.get(aa, 0.0), MW.get(aa, 0.0), CH.get(aa, 0.0)]
        rows.append(row)
    return np.array(rows, dtype=float)
```

Here is the part that took me a while to see. Chapter 1 never made this choice at all. A tool called AMRFinderPlus handed me a finished table of genes present or absent and substitutions found or not. Somebody else had decided what counted as a feature. I inherited that decision and never noticed I had.

A feature matrix is not a fact about nature. It is a claim about what matters.

2.3 HOW A TREE CHOOSES, AND WHAT STOPS IT

Every textbook gives the same rule for HLA-A*02:01. Leucine or methionine at position two. GILGFVFTL has neither. It binds anyway, and it binds so well that your killer T cells go after it before almost anything else in the virus. The rule is not wrong. It is just not the whole story.

Now watch what happens when a machine tries to learn that rule from data.

A decision tree gets all 180 columns, and it may ask only one of them first. It chooses by counting.

Put every measurement in one pile. 15,597 laboratory measurements on this molecule, 8,376 counted as binders and 7,221 not. The pile is a coin flip. Now ask one question and let the pile fall into two. Ask something irrelevant and nothing happens, because

both piles come out half and half and you have learned nothing. Ask about leucine at position two and one pile should come out mostly binders. The tree tries all 180 questions, measures how much each one tidies the piles, and keeps the winner. Then it starts the whole search again inside each pile.

Impurity. How mixed a pile is. A question is scored by how much it lowers the impurity of the two piles it creates, and the tree keeps the question with the largest drop. The trees in this chapter measure impurity with the Gini index, which is scikit-learn's default.

The tree cannot look ahead. It takes whichever question pays now. A weaker first question might set up a far stronger second, and the tree never looks. That is called a greedy search. Most of the time it works, because position two really does carry signal on its own, so the tree will find it.

And then the tree's first question does exactly what the textbook does. It sends GILGFVFTL to the wrong side.

One awkward peptide would be a footnote. Sidney and colleagues counted fourteen A*0201 epitopes in vaccinia virus and reported in 2008 that eight of them did not fit the rule (Figure 2.4).

Chapter 1 never had this problem. One question about one position in *Campylobacter* was correct for 99.67% of the 3,984 isolates I tested. One question, whole model, done.

Peptides do not hand you that. So the tree asks again. And again. Nothing in the arithmetic tells it when to stop.

Ask enough questions and every peptide ends up alone in its own group. A group of one is never wrong. A near-perfect score on all 15,597 measurements, and worth nothing. A group holding one peptide is not a rule. It is a memory. The tree has recorded that this exact sequence bound. It has learned nothing about a sequence it has never seen.

Overfitting. A model that fits the data in front of it and describes nothing else.

So what stops the tree? You do. You can cap how many questions deep it may go. You can refuse to split a group below some size. Or you can let it grow wild and cut the branches back afterwards, which is called pruning. Every one of those is a number you picked. Not one of them came out of the data.

The honest way to choose is to hide some peptides, fit the tree on the rest, then ask it about the ones it never saw, and keep the depth that does best on those. Then do it again with a different part held back, five times over, so that every peptide is held back

once, and average the five scores. That is cross-validation.

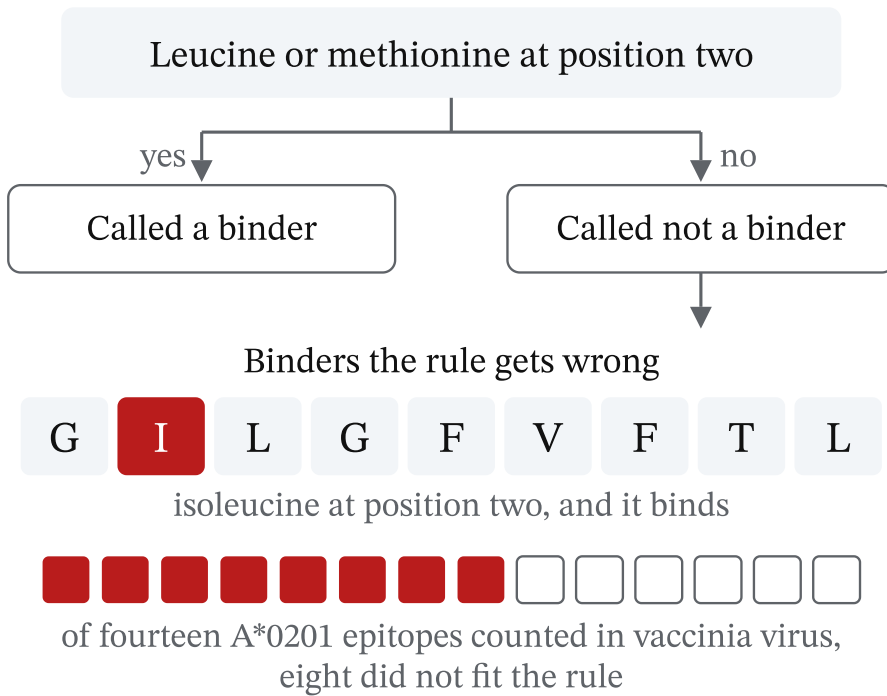


Figure 2.4 The textbook rule for HLA-A*02:01 answers no for GILGFVFTL, which carries isoleucine at position two and binds anyway, and for eight of the fourteen A*02:01 epitopes counted in vaccinia virus.

2.4 THE RULES

Here is what was fixed, and what I changed after looking.

RULES FOR THIS CHAPTER

Fixed before any modelling.

One allele, HLA-A*02:01. Peptides nine amino acids long. A measurement below 500 nanomolar is a binder. Three encodings, compared on the same peptides with the same model. Split by source protein, not by row. Depth chosen by grouped cross-validation over depths one to twenty, five folds each, keeping the depth with the best cross-validated F1, the score that combines precision and recall.

Changed after looking.

Keep only the measurements reported in nanomolar. The database holds half-lives, melting temperatures and interatomic distances in the same numeric column. Match the allele name exactly instead of asking for every name that contains 02:01, because the substring admits molecules from other species and eleven engineered point mutants of the human one.

2.5 WHERE THIS COULD GO WRONG

Cross-validation can still be fooled, and here the data does the fooling.

There are 15,597 measurements and only 9,142 distinct peptides, so 6,455 of those measurements are a repeat reading of a peptide already in the set (Figure 2.5). Hide a peptide from the tree and a copy of it may still be sitting in the data you trained on. The tree is not being tested then. It is being asked to recall. The repair is to collapse the repeats before anything is split, one row for each distinct peptide carrying one label. Choosing that label is a problem of its own, and it comes later in this chapter.

Chapter 1 had the same problem and got away with it. 344 groups of closely related *Campylobacter* isolates sat on both sides of a random split, and it changed nothing: the one question about position 86 was still right 99.65% of the time. The model had found the one position that decides resistance, and there was nothing left worth memorising. A tree deep enough to memorise has plenty to memorise.

15,597 measurements

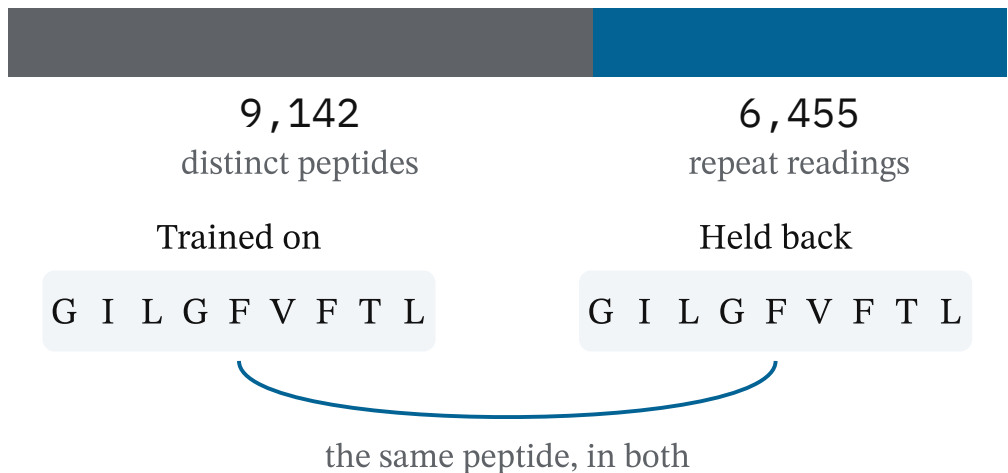


Figure 2.5 Of the 15,597 measurements in the first cohort, 9,142 were distinct peptides and the other 6,455 were repeat readings, so a peptide held back from the tree could have a copy of itself in the data the tree was trained on.

Then comes the harder version. Not copies. Relatives.

Change one letter in GILGFVFTL. Is it still the same peptide? Ask the MHC molecule holding it and the answer may be yes, because the groove grips position two and position nine, and if you change position five, the grip barely notices. Ask a T cell and

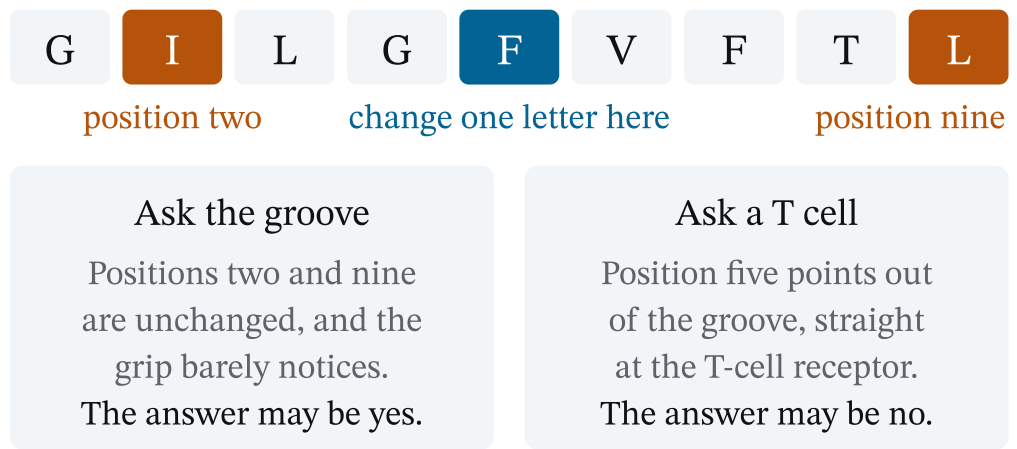
the answer may be no, because position five points up, out of the groove, straight at the T-cell receptor. Same nine letters, two different answers, depending on who is asking (Figure 2.6).

Those 9,142 peptides came out of 2,874 different proteins, just over three peptides per protein. Split them at random and peptides from the same protein land on both sides of the line. The model studies one. Then it gets asked about the neighbour.

The repair is to split by group instead of by row. Every peptide from one protein goes to the same side, training or testing, never both. That is grouped splitting. One line of code, and the accuracy usually drops. That drop is the point. The old number was scoring the model on neighbours it had already studied, and the new one asks about proteins it has never met.

Grouping by source protein is a guess. Two peptides from different proteins can differ by a single letter, and two from the same protein can share almost nothing. The protein is a convenient handle rather than the thing that matters. What matters is how similar two sequences are, and the usual fix is to cluster the peptides by sequence identity and split whole clusters instead. Then you have to choose the identity threshold. Eighty per cent? Seventy? You have swapped one arbitrary line for another.

So you decide, and then you report a number that is partly a report about your decision. There is no neutral split.



Two different answers, depending on who is asking.

Figure 2.6 One letter changed at position five, with the two gripping positions untouched. The groove may hold the peptide as before, while the T-cell receptor, which position five points straight at, may not. Same nine letters, two answers.

2.6 A COLUMN WITH NO UNIT

A 45-degree melting point is not a binding affinity.

IEDB, the database that supplies this data, stores measurements from many assay types in the same column. Radioactive inhibition assays report affinity in nanomolar. Thermal shift assays report melting temperatures in degrees. Stability assays report half-lives in minutes. Structural assays report distances in ångströms. The column is called quantitative measurement. No unit is attached.

Quantitative measurement. The single numeric column IEDB uses for every assay type, with the unit kept somewhere else.

A 500-nanomolar threshold on that column does not select binders. It selects numbers below 500.

Of the 15,597 measurements, 3,874 were not nanomolar, and of those, 3,418 fell below 500 and were labelled binders. Among those 3,874 were 2,225 half-lives, 1,087 unitless figures, 333 melting temperatures and 219 interatomic distances. A half-life of 23 minutes is a stable complex, not a tight binder, and it was sitting in the training data as one.

The corrected cohort is 11,723 measurements, 8,346 distinct peptides, and 4,958 that bound below 500 nanomolar. That is 42.29%, not 53.7%. The count of repeat readings falls with it, from 6,455 to 3,377.

2.7 WHAT THE TREE FOUND

In 1991, somebody stripped the peptides off an MHC molecule and sequenced them all at once. The pattern was unmissable. Two positions out of nine did nearly all the gripping.

The tree found the same two positions. Nobody told it they existed.

The grouped split put 6,971 of the 8,346 peptides on the training side and the other 1,375 on the test side. The tree was given those 6,971 training peptides, nine letters each, and the same 180 columns. That is all. No groove, no pockets, no anchors. Nine letters and a label saying bound or did not.

Its first question was position two, leucine. Then position two again, methionine. Then position nine, valine. Then position two, isoleucine. Then position nine, leucine.

Five questions. Two positions. It had nine to choose from.

Those two are the pockets. An MHC molecule is the clamp on your cell surface, and the second and ninth residues drop into it and hold the fragment still. The tree found the clamp by counting.

It did not discover it. Falk and colleagues published the pattern in *Nature* in 1991, thirty-five years ago.

A tree insisting that position four decides everything would have told me the pipeline was still broken: contaminated units, duplicated peptides, relatives sitting on both sides of a split. Any of those shows up as a model confidently finding the wrong thing. It found the right thing instead.

Chapter 1 ended the same way. The model walked to the one position in *gyrA* that microbiologists have known about for decades. Recovering a known answer is a receipt, not a finding.

2.8 WHAT A SCORE IS WORTH

I can build you a model that is right 63% of the time and never finds a single binder. It answers no to everything.

37.2% of the 1,375 peptides in the test set bind. Say no to all of them and you are correct about the other 62.8%. Accuracy 0.6284. Binders found: none.

That 37.2% is a smaller share than the 42.29% above because this one counts peptides, with every repeat reading collapsed into one label.

Accuracy counts how often you were right. It says nothing about what you were right about.

So you need two numbers instead of one. Of the peptides you called binders, how many actually bound? That is precision. Of the peptides that actually bound, how many did you catch? That is recall.

Either one alone is easy to fake. Call a single peptide a binder and be right about it, and precision is perfect while recall is almost nothing. Call everything a binder, and recall is perfect while precision collapses to the share that bind. F1 is one number that only rises when both rise. The model that says no to everything scores 0.0000.

Now the comparison. Every number below comes from the same held-out proteins.

The textbook rule first. Leucine or methionine at position two, predict binder. Accuracy 0.7542. F1 0.6667. One question about one position, and it beats saying no to everything on both counts.

Then the trees, tested on the same proteins they never saw during training. Nine columns of alphabet numbering, F1 0.6983. 180 columns of one-hot, F1 0.7153. 27 columns of hydrophobicity, molecular weight and charge, F1 0.7399.

So yes. The tree beats the rule. One hundred and eighty columns and thirteen levels of questions bought 0.0486 F1 over one rule about one position, and the 27 columns of chemistry bought 0.0732. Chapter 1 went the other way, where eighty-four features bought nothing. Here the model wins, and that is the size of the win.

I expected one-hot to win. It did not. I am saying so after the scores, and without the note I said I would write, so it is not a prediction.

2.9 WHY THE SMALLEST ENCODING WON

To one of my encodings, leucine and isoleucine were neighbours. To another they were strangers. That difference decided which model won.

I wrote down the cost of one-hot as a caveat before anything ran. It turned out to be the result.

Look at how deep each tree had to go. Chemistry needed seven levels of questions. One-hot needed thirteen.

That is the mechanism. A tree using chemistry asks one question. Is this position hydrophobic. That catches leucine, isoleucine, valine and methionine at once. A tree using one-hot has to ask about each of them separately, four questions to say what the other tree said in one.

The MHC groove does not read letters. It has a pocket with a shape and a chemistry, and any residue that fits, fits. An encoding that describes shape and chemistry sits closer to what the molecule is doing than one that spells names.

One caution before anyone generalises this. Three physicochemical properties per position is a choice I made before seeing any result, and a different three might do better or worse. This is one comparison, on one allele, with one model.

2.10 THE DEPTH I REPORTED

Eight questions deep scores a cross-validated F1 of 0.7095. Thirteen questions deep scores 0.7118. Both are the one-hot tree. I reported thirteen. I am not certain that was right.

A decision tree needs a depth limit. Without one, the one-hot tree kept splitting until every group agreed with itself, which took 1,140 groups for 6,971 training peptides, and its accuracy on them was 1.0000. On peptides it had not seen, that same tree scored 0.7789 for accuracy, and the one I kept scored 0.7927.

So I tested every depth from one to twenty, five times each, on peptides held back from different proteins each time, and kept the depth with the best cross-validated F1. Thirteen won (Figure 2.7).

The sweep ran over all 8,346 peptides, and the training and test split was drawn after it, so the 1,375 test peptides sat inside the folds the depth was chosen on. The trees scored earlier in this chapter were fitted on the training peptides alone, and every fold of the sweep held its own peptides back while its tree was fitted, but the depth those trees were given had been chosen over a cohort that included the test peptides. Of everything I report, the depth is the one choice the test set helped make.

Now look at the curve it won on. The score climbs steeply to about depth eight. After that it is almost flat. Depth eight to depth thirteen buys 0.0023. Five extra levels of questions for two thousandths.

There is a standard rule for exactly this situation, and it exists because a flat plateau is mostly noise, and the peak of noise is not a real peak. Under that rule I would have reported depth eight: a tree half as deep, easier to read, almost the same score.

The one-standard-error rule. Take the simplest model within one standard error of the best one.

So why did I report thirteen? Because the rule I wrote down beforehand said take the maximum, and I only saw the plateau afterwards. Changing the rule after seeing the answer is how you get results that do not replicate. Even when the new rule is better. Even when you are certain your motives are clean.

Both numbers are in the repository. Depth thirteen is the reported one because it is the pre-specified one.

Notice what this is not. It is not a data problem, or a modelling error, or something a better pipeline would catch. It is a judgement, made by me, that changes the number I publish. There is no neutral depth either.

Eight questions deep



Thirteen questions deep



Cross-validated F1 for the one-hot tree

Five extra levels of questions for 0.0023

Figure 2.7 Two points on the one-hot tree's cross-validation curve, eight questions deep at 0.7095 and thirteen questions deep at 0.7118. The sweep tested every depth from one to twenty, five times each on peptides from different proteins, and thirteen took the best score.

2.11 WHICH MISTAKES IT MAKES

My best model missed 117 real binders. My second best missed 153. There are 511 binders in the test set, so those are both bad numbers. They are bad in different ways, and F1 cannot tell you how.

1,375 peptides. 511 bind. 864 do not.

The chemistry model called 554 of them binders. 394 of those calls were right and 160 were wrong. The one-hot model was more cautious. It called 490 binders, and 358 were right. Being cautious cost it: it missed 153 real binders where chemistry missed 117. So chemistry catches 36 more binders than one-hot and raises 28 more false alarms doing it. That is the trade underneath a gap of 0.0246 in F1.

Precision: chemistry 0.7112, one-hot 0.7306. Recall: chemistry 0.7710, one-hot 0.7006. One-hot is the better guesser. Chemistry is the better finder.

Which one you want depends on what a mistake costs, and nothing in the data can tell you that. A false alarm is one assay you run and throw away. A missed binder is a peptide you never look at again. For screening vaccine candidates, those two prices are nowhere near equal. Twenty-eight more assays would be cheap. Thirty-six peptides you never test could include the one that mattered.

Now the textbook rule, on the same 1,375 peptides. Leucine or methionine at position two catches 338 binders and raises 165 false alarms (Figure 2.8). Chemistry catches 56 more than that, and raises 5 fewer. Better on both counts at once. No trade, no

judgement call, just better.

That is the clearest thing this chapter has produced, and it took 27 columns and seven levels of questions.

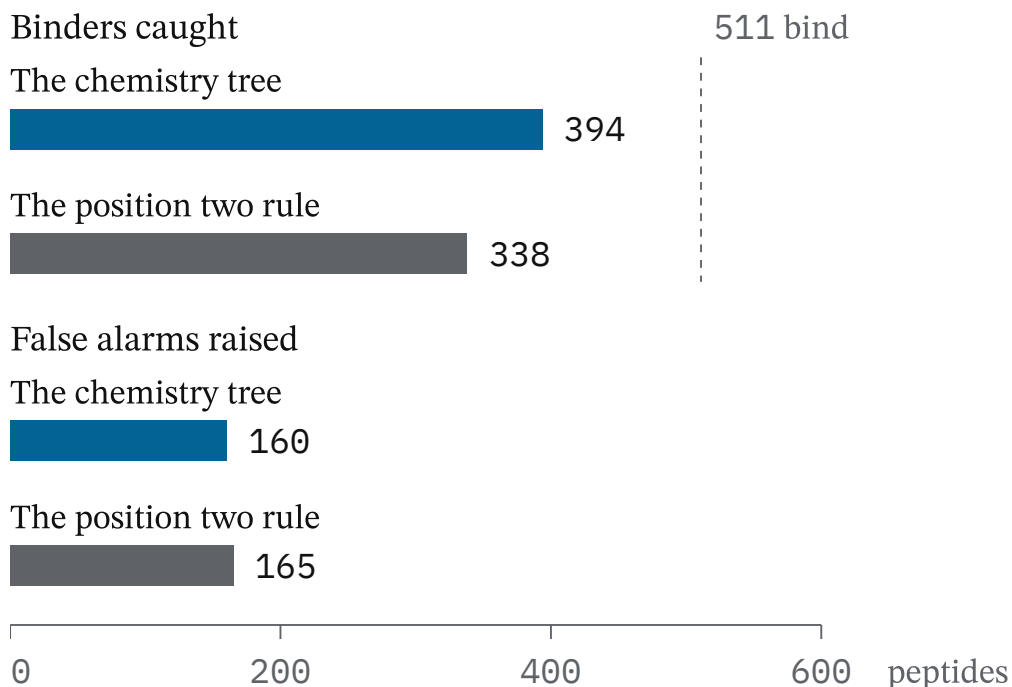


Figure 2.8 Of the 511 binders in the test set, the chemistry tree caught 394 and the position two rule caught 338. The tree raised 160 false alarms and the rule 165, both counted on the same held-out proteins.

2.12 PEPTIDES THAT DISAGREE WITH THEMSELVES

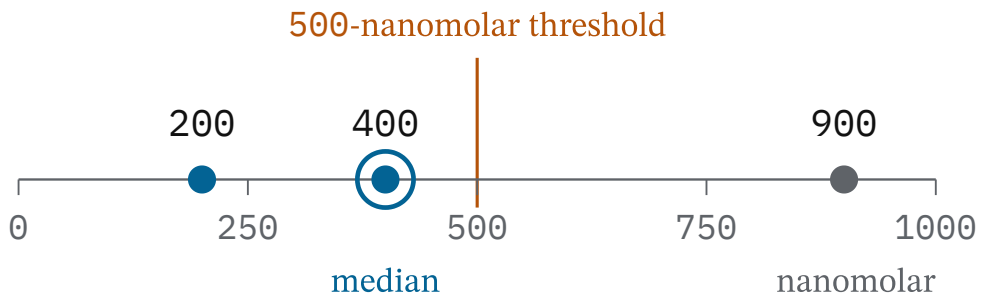
127 peptides in this dataset disagree with themselves. They were measured more than once, and one result says binder while another says not. I expected that number to be far worse.

8,346 distinct nine-mers sit in the corrected cohort, and 2,660 of them were measured more than once, in different labs, in different years, with different reagents. Nearly a third of the dataset. For most of them the repeats agree. Two readings of 15 and 40 nanomolar both say binder. Two at 6,000 and 12,000 both say not. 127 do not agree, because one reading falls below 500 nanomolar and another falls above it. Same nine amino acids. Same molecule. Two answers. That is 4.8% of the repeat-measured peptides.

So what do you do with 127 peptides that contradict themselves? You cannot keep both readings, because the model would see the same nine letters labelled two ways and learn nothing from either. You cannot drop them either, because that deletes the peptides sitting nearest the line, and those are the ones a binding model most needs to get right.

I took the median of every peptide's measurements and labelled that. Three readings of 200, 400 and 900 become 400, and 400 is a binder (Figure 2.9). It is a defensible choice. It is still a choice, and a different one would give a different training set.

Now the part worth sitting with. A binding affinity comes out of an assay, run by people, with reagents that vary. Run it twice and you get two numbers. Draw a line anywhere through the middle of them and some pairs will land on opposite sides. Those 127 peptides are what measurement looks like when you stop rounding it off. Move the threshold and a different set of peptides straddles it. The disagreement does not go away. It relocates.



The median is 400, so the peptide is labelled a binder.

Figure 2.9 Three readings of one peptide, at 200, 400 and 900 nanomolar, against the 500-nanomolar threshold. Two fall below it and one above, and the median, 400, is the label the peptide was given.

2.13 THE ALLELE FILTER

My human dataset contained 263 measurements from a rhesus macaque. Also horse. Also cattle. Also chicken.

None of that was a download error. I wrote one line of code that let them in.

The allele I am studying is HLA-A*02:01. To pull its measurements out of a nine-gigabyte file, I asked for every row whose allele name contains 02:01. That is the natural thing to write, and it is wrong.

Mamu-A1*002:01 is a macaque molecule. Eqca-1*002:01 is horse. BoLA-3*002:01 is cattle. Gaga-BF2*002:01 is chicken. All of them contain 02:01, because the naming is done per species. The number after the asterisk identifies an allele within its own species, not across them. Two different species can both have an 02:01 and share nothing but the label.

Then there were eleven more. HLA-A*02:01 K66A and HLA-A*02:01 A150P are point mutants, engineered in a lab, one amino acid changed inside the groove. Those are human, and they are the same allele, and they are still the wrong molecule. The whole point of mutating position 66 is to change what binds there.

Together they added 302 measurements and 262 peptides that do not belong. Matching the allele name exactly removed all of them and reproduced my feasibility scan to the row. The cohort of 11,723 is what the exact match leaves, so none of those rows are in it. Listing 2.2 shows the filter.

Listing 2.2 The exact allele match, from pipeline.py.

```
ALLELE      = "HLA-A*02:01"  # Exact string, deliberately not a substring match.
                                # Matching on "02:01" also picks up Mamu-A1*002:01
                                # (macaque), Eqca-1*002:01 (horse), BoLA-3*002:01,
                                # Gaga-BF2*002:01 and eleven point mutants of
                                # HLA-A*02:01 itself (K66A, A150P and so on). Those
                                # are different grooves on different molecules.
```

Unit contamination announced itself. Melting points and half-lives sit in the wrong column, and a check catches them. This did not announce itself. Every one of those rows is a real peptide, measured properly, against a real MHC molecule. Nothing is corrupt. They are simply not the molecule I said I was studying, and a model would have learned from them happily. The nomenclature was doing exactly what it was designed to do, and my filter was the thing making the claim.

A substring is not an identifier.

That is where this chapter ends. Not on the model that worked, but on the filter that almost let a macaque define a human molecule. Everything is in the repository: the corrected cohort, the three encodings, every metrics file and every number I got wrong on the way. Chapter 3 asks what happens when two annotation tools read the same genome and disagree about what is in it.

This chapter is rewritten from the posts for Days 12 to 25 of the series.

REFERENCES

- Falk, K., Rötzschke, O., Stevanović, S., Jung, G. and Rammensee, H.-G. (1991). Allele-specific motifs revealed by sequencing of self-peptides eluted from MHC molecules. *Nature*, 351(6324), 290-296. <https://doi.org/10.1038/351290a0>
- Sidney, J., Assarsson, E., Moore, C., Ngo, S., Pinilla, C., Sette, A. and Peters, B. (2008). Quantitative peptide binding motifs for 19 human and mouse MHC class I molecules derived using positional scanning combinatorial peptide libraries. *Immunome Research*, 4, 2. <https://doi.org/10.1186/1745-7580-4-2>

RUN IT

Repository github.com/Qasim-Hussain-Code/peptide_mhc_binding_prediction
Commit [d701b62eb203](https://github.com/Qasim-Hussain-Code/peptide_mhc_binding_prediction/commit/d701b62eb203)

CITE THIS CHAPTER

Hussain, Q. (2026). What does a cell show to the immune system? In *Machine Learning for Biology*, chapter 2.

IS A GENE A FACT OR A CALL?

Model: Random forest

When two standard tools read the same bacterial genome and disagree about a gene, can that disagreement be predicted from the DNA alone?

SUMMARY

Bakta and Prokka were run on the same 25 bacterial genomes, one from each of 25 species, and they almost never disagreed about where genes are. Only 72 of 87,960 Bakta gene calls had no matching Prokka call, because both tools find genes with the same underlying program. They disagreed far more about what the genes are: of 50,210 genes that both tools named, 51.7% received different names. A random forest predicted those naming disagreements with a Matthews correlation coefficient (MCC) of 0.303 on five held-out genomes, but its signal came from what each tool found in its reference databases, and with only DNA features it scored 0.142 on the same genomes and 0.070 in cross-validation on the twenty training genomes.

In 2000, a team sequenced *Campylobacter jejuni* and reported 1,654 genes. In 2007, a team that included two of the original authors went back to the same genome and reported fewer. There was no new strain and no new sequencing. The same 1,641,481 letters were read again, gene by gene, by curators. The DNA did not change. The count did (Figure 3.1).

Somebody printed a microarray with 1,731 spots on it, one for every gene first identified in the genome: real glass, real DNA, an experiment you could run. By the time the genome paper was published, 77 of those spots stood for genes that were no longer in the annotation. The count had been revised down to 1,654, and the array had been built from the longer list. The 77 spots stayed on the glass, and the gene lists in the analysis software flagged each one as a “not annotated gene”. Seven years later the count went to 1,643. From 1,731 to 1,654 to 1,643 is eighty-eight fewer genes in the list.

Microarray spots, one per gene in the first list



Genes in the genome paper, 2000



Genes after the same genome was read again, 2007



Genes counted in the same 1,641,481 letters

Figure 3.1 One *Campylobacter jejuni* genome of 1,641,481 letters, counted three times, with each bar drawn from zero: 1,731 spots on a microarray built from the first gene list, one for every gene in it, then 1,654 genes in the genome paper of 2000 and 1,643 in 2007. The open end of each later bar marks how far the count had fallen from the first, 77 by the time of the paper and eighty-eight by 2007.

The same thing still happens every day. Bakta and Prokka are two standard annotation tools, both widely used. Give them the same bacterial genome and, at some places, one says there is a gene here and the other says there is nothing here. Same DNA, the same letters in the same order, read twice. Neither program is broken.

This chapter asks whether that disagreement can be predicted from the sequence alone. It does not ask whether a gene is really there. Nothing in this project can tell you that. The answer being predicted, which machine learning calls the label, was produced by software, so a model trained on it learns the annotator rather than the organism. That is the harder thing about this chapter, and it is the point.

3.1 WHERE GENES COME FROM

Start with what a gene caller is looking for. A protein-coding gene begins with a start codon, runs in groups of three letters and ends at a stop codon. The stretch in between is an open reading frame. Find every open reading frame and you have found every possible gene. That is the easy part, and on its own it is useless.

Three of the sixty-four possible codons mean stop. *Campylobacter* is AT-rich: most of its letters are A or T, and the three stop codons are written mostly in those two letters, so in its genome they are common. Read a random stretch and you hit a stop every thirteen

codons or so. So the genome is full of short open reading frames that mean nothing: millions of letters, six reading frames, stops everywhere. The real genes average 316 codons. They are the long ones. But length alone will not separate them, because a short gene is still a gene and a long accident is still an accident.

So the caller weighs other things. Does the codon usage inside this frame look like the codon usage of known genes in this organism? Is there a ribosome-binding site sitting just upstream? Does the protein it would produce resemble a protein somebody has already described? Each of these is evidence. None of them is proof. And each needs a cut-off: how similar is similar enough, how long is long enough, how close the binding site has to sit. Nobody finds those numbers in the genome. They are chosen by the people who write the software, and two programs can choose them differently. That is the whole of this chapter.

Of the *C. jejuni* genome, 94.3% codes for protein. The other 5.7% is about 93,564 letters, and it is neither empty nor junk. Some of it is genes that make RNA instead of protein: the ribosomal RNA the cell builds its ribosomes from, and the transfer RNAs that carry amino acids into place. Some of it is control: promoters where transcription starts, ribosome-binding sites, terminators that say stop here. Besides ribosomal and transfer RNA, this genome has four annotated RNAs. Four, in 1.6 million letters. That is a count of how many anybody has written down, not a claim that only four exist.

Which brings me to the word intergenic. It sounds like a description of a place, between the genes. What it actually means is that nothing has been annotated here. It is a residual, defined by absence, and the absence is in the record rather than in the molecule. Move a gene boundary and the intergenic space moves. Withdraw a gene and the intergenic space grows. When two annotation tools disagree about a region, one of them calls it a gene and the other calls it intergenic, and neither is looking at anything the other cannot see.

3.2 WHY A RANDOM FOREST

The model in this chapter is a random forest: many decision trees, each a chain of yes-or-no questions learned from a different slice of the data, voting on the answer.

Chapter 2 was built on a single tree, and the best moment in it was reading the thing. I could print the questions it had chosen, and it had found a real binding rule that nobody told it about. A forest gives that up. Three hundred trees will score higher than one, and I will not be able to read any of them. So this chapter has a harder label and a model I cannot read.

3.3 WHAT THE TWO TOOLS ACTUALLY DO

Bakta and Prokka do not disagree the way I assumed. I assumed two independent gene callers, each trained differently, each finding its own answer. That is not what is happening underneath.

Prokka calls genes with a program called Prodigal. Bakta calls genes with a program called Pyrodigal, and Pyrodigal is a rewrite of Prodigal in a different language: same model, same scoring, faster code, some bugs fixed. The two are close enough to the same caller that most of a genome will come out identical either way.

The disagreement comes from two decisions about what to look for. Bakta adds a search for very short proteins, under thirty amino acids, which Prodigal was never built to find. Bakta also calls pseudogenes: genes broken by a stop codon or a frameshift, still recognisable, still worth recording. Prokka mostly does not. So a real disagreement between them will rarely look like two tools reading the same DNA and reaching different conclusions. It will look like one tool asking a question the other tool never asked.

That changes what the forest should find. If the two callers were independent, disagreement might trace to something subtle in the sequence, a signal one caller catches and the other misses. If they mostly share a caller, disagreement should trace to something blunter. Length, mostly, because short proteins are what one tool looks for and the other does not. If the forest's top feature, the column of data it leans on most, is not length, I have misunderstood something.

3.4 THE RULES

The rule for what counts as a disagreement was written into the code before the label was built. Each protein-coding gene Bakta called is one row. If anything Prokka annotated on the same strand overlaps that call, by as little as a single base, the call counts as matched. If nothing does, Bakta made a call there and Prokka did not, and the row is a disagreement.

One base sounds loose, and it is meant to be. The two tools mostly agree on where a gene starts and stops, and when they do not, it is usually over the exact start codon: a difference of a few dozen letters, inside a call both tools clearly made. That is not a disagreement. Both tools found the gene. A rule that demanded a large overlap on both sides would turn some of those start-codon differences into missing genes. So the label

asks only whether Prokka claimed the stretch at all. Whether the two calls share a start, and whether they share a stop, are recorded beside the label rather than mixed into it (Figure 3.2).

Strand matters. DNA has two strands, and a gene can sit on either. Two genes on opposite strands can occupy the same letters and still be different genes, so only what Prokka annotated on the same strand is consulted. If the only call Prokka made at that place is on the other strand, the Bakta call counts as one Prokka did not make. The kind of annotation does not matter. If Prokka called a transfer RNA, or anything else that is not a protein-coding gene, on the same strand where Bakta called a protein-coding gene, Prokka did not leave the region empty. It made a different claim, not a missing one, and counting it as silence would put two unlike things in one label. Those calls count as matched, and there are 29 of them.

One more decision was easy to get wrong. When both tools call the same gene but pick different start codons, which coordinates go in the table? Every row carries Bakta's, matched or not: where Bakta's call starts and stops, and how long it is. How much of it Prokka's call covers is kept as a separate count. So the length of every call in the table is the length Bakta chose, and a model that leans on length can learn that tool's habits instead of the sequence.

The rule did not change once the label existed, and that matters more than it looks. A threshold chosen after seeing which one gives the better story is not a threshold. It is the story wearing a number.

Later in the chapter the question moved from where genes are to what they are called, and that comparison needed a rule of its own. Comparing product names, the names each tool gives to the protein a gene makes, by exact string match is too strict. One tool writes 50S ribosomal protein L7/L12, and the other writes the same thing with a bracket on the end. Those are the same claim, and counting them as different would make the whole exercise meaningless. So names are normalised before comparison. Strip a trailing bracket. Strip a trailing gene symbol, because one tool often appends it and the other does not. Strip a leading hedge word: putative, probable, possible, predicted or conserved. Then ignore capitals and treat punctuation as a space. Under that rule, putative cytosol aminopeptidase and Cytosol aminopeptidase are the same name. The check on that rule was fixed before I ran it.

RULES FOR THIS CHAPTER

Fixed before the label was built.

Each protein-coding gene Bakta called is one row. It counts as matched if anything Prokka annotated on the same strand overlaps it by at least one base, whatever the kind of annotation; if nothing does, it is a call Prokka did not make. Prokka's annotations on the opposite strand are not consulted. Every row carries Bakta's coordinates. The overlap with Prokka's call, and whether the two calls share a start and a stop, are recorded beside the label, not in it.

Fixed before any model was fitted.

Names are compared after stripping a trailing bracket, a trailing gene symbol and a leading hedge word such as putative, then ignoring capitals and treating punctuation as a space. The result is checked against the strings as written, ignoring only capitals, spacing and a final full stop, and against the loosest rule I was willing to defend, which also drops generic words and ignores word order. Both are checks, not alternatives to switch to.

Changed after looking.

The gene-symbol strip first used each record's own symbol field. It now builds one list of symbols per pair, from both tools, and applies it to both sides. Identical raw strings can never be scored as a disagreement, and the pipeline checks this on every run. After a silent failure inside Prokka, the pipeline also records how many of each kind of annotation, such as genes or ribosomal RNAs, each tool produced, per genome.

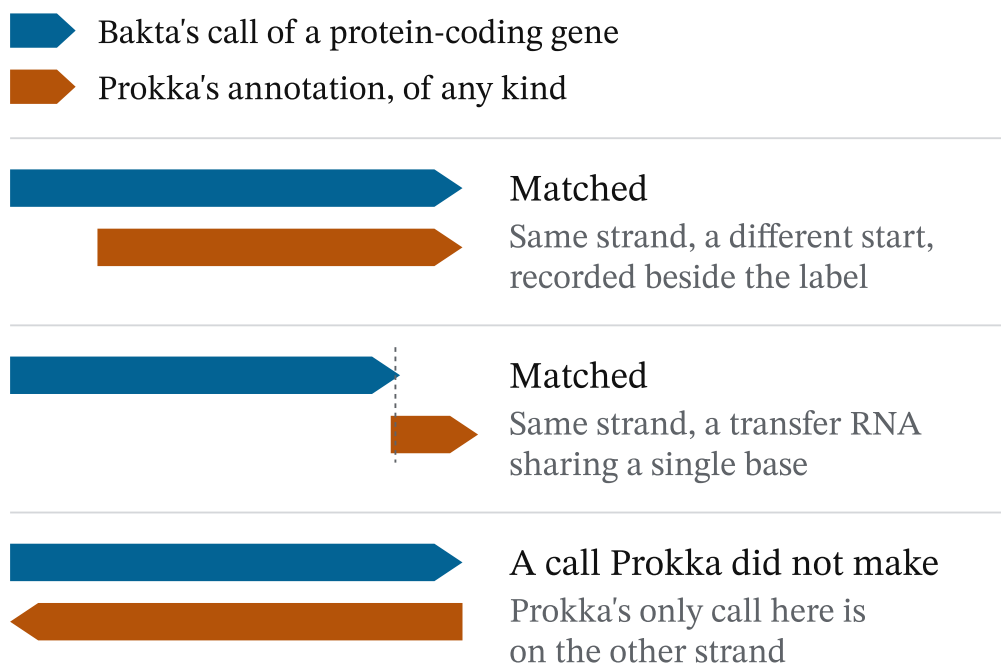


Figure 3.2 The rule that built the label, in three cases, with each arrow pointing along the strand it sits on. A Bakta call that shares even one base with anything Prokka annotated on the same strand is matched, whether Prokka's call starts somewhere else or is another kind of annotation, such as a transfer RNA; a Bakta call with nothing from Prokka on its strand is a call Prokka did not make, whatever Prokka called on the other strand.

3.5 SEVENTY-TWO ROWS

The pipeline ran on 25 bacterial genomes, one from each of 25 species spread across eight phyla. The *Campylobacter jejuni* genome this chapter opened with is one of them. Across all 25, Bakta made 87,960 gene calls, and 72 of them have no matching call from Prokka. That is a disagreement rate of 0.0008, eight in ten thousand. Of the 87,888 calls both tools made, 87,788 have identical start and stop coordinates, to the base.

The reason was knowable before I downloaded a single genome. With Prokka on Prodigal and Bakta on a rewrite of Prodigal, asking whether Bakta and Prokka disagree about where genes are is mostly asking whether Prodigal agrees with itself. It does. Leave aside the 43 calls Bakta made with its search for very short proteins, and in the other 87,917, there are 38 divergences, which is 0.04%. I expected a small signal. I got 72 rows.

How much overlap the rule demanded hardly mattered. When 87,788 of the 87,888 matched calls are identical to the base, they would match under any threshold, and the 72 with nothing overlapping them would match under none. A stricter threshold could have changed the label for at most the remaining hundred of the 87,960 calls. I could not have known that in advance, and choosing the threshold afterwards, once I could see which one flattered the result, would not have been a choice at all. A rule that turns out not to bind is still worth fixing beforehand.

The reasoning about length held. Asked to pick out the 72 coordinate disagreements, the forest leaned on the length Bakta gave each call more than on anything else. Of those 72, 34 come from one module. After the main gene caller finishes, Bakta runs a second search for short open reading frames, under thirty amino acids. Prokka has nothing equivalent. Bakta made 43 such calls across the whole panel, and 34 of them are disagreements. Nearly every short protein Bakta finds, Prokka misses, because it never looked. Prodigal's default minimum gene length is 90 base pairs, which is thirty codons. The shortest disagreement here is 45 base pairs.

Now the part that is actually biology. Look at what those short proteins are. They are Type I toxin-antitoxin system Ibs family toxins, and leader peptides from the tryptophan and histidine operons. These are real, well described and functionally important. The first is a toxin whose antitoxin is an RNA that stops it being translated. The second is a short peptide whose ribosome stalls or does not, and in stalling decides whether the operon behind it gets transcribed. This is textbook regulatory biology, and exactly the class a dedicated short open reading frame finder is built to catch.

So the disagreement is real, and it points at something real. But the label is not what I said it was. At the start of this chapter, I described two tools reading the same DNA and reaching different conclusions. What the data holds is one tool running an extra module. That is a fact about software architecture, and the biology is downstream of it.

3.6 TWO FAILURES THAT EXITED CLEANLY

One more thing had gone wrong before I could trust any of this. Prokka found zero ribosomal RNAs in all 25 genomes. It reported that, and exited successfully.

Every bacterial genome has ribosomal RNA. It is not optional, and a genome with none is not a genome. The tool did not crash, did not warn and did not return an error code. It printed Found 0 rRNAs and finished. I found it while the annotation was still being set up, before any label had been built.

Here is the chain. Prokka does not find ribosomal RNA itself. It calls a program called barrnap. Barrnap does not do it either: it calls nhmmer. The nhmmer binary in that environment had been compiled against one version of a maths library, and the environment shipped a different version. So nhmmer failed at runtime, every time. Barrnap got nothing back and reported nothing. Prokka reported what barrnap told it. Three programs deep, and nobody raised a hand (Figure 3.3).

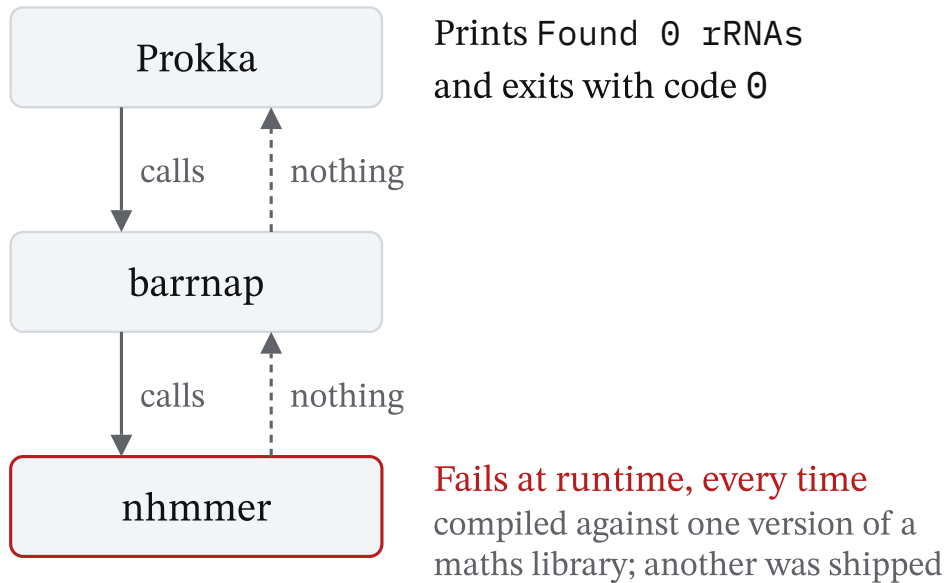


Figure 3.3 How Prokka reported zero ribosomal RNAs and exited with code 0. Prokka calls barrnap and barrnap calls nhmmer; nhmmer had been compiled against a different version of a maths library from the one shipped and failed every time, and no error came back up the chain.

Think about what that would have done to the comparison. A disagreement is a region one tool called and the other did not. If Prokka finds no ribosomal RNA anywhere, then every ribosomal RNA Bakta calls looks like one. Not a real one: a missing shared library, which would have gone into the data as biology.

I caught it because the count was zero rather than low. Zero is impossible, and impossible is visible. The first complete Prokka run, 87,901 gene calls and no ribosomal RNA, was thrown away and redone. The rule I took from it is that a program exiting successfully is not evidence that it did its job. The pipeline now records how many of each kind of annotation each tool produced, per genome. I do not expect this bug again. I expect the next one to exit zero as well.

With the coordinates settled, the same regions could be asked a different question. At 87,859 places, both tools called a protein-coding gene, almost always with the same start and stop. Do they agree on what those genes are? The second failure belongs to that question, and this time the code was mine.

My code decided that GTPase Era and GTPase Era were different names, 23 times (Figure 3.4). Same string: same letters, same spaces, same capitals, scored as a disagreement between two tools that had written exactly the same thing. The way it happened is common.

The gene-symbol strip is where it broke. To strip a trailing gene symbol, you need to know what the gene symbols are, and my first version read each record’s own symbol field and stripped that. Bakta fills that field on 19,216 of these regions. Prokka fills it on 48,201. So for thousands of pairs the strip fired on one side and not the other. Two identical strings went in, one came out shortened, and the comparison said different.

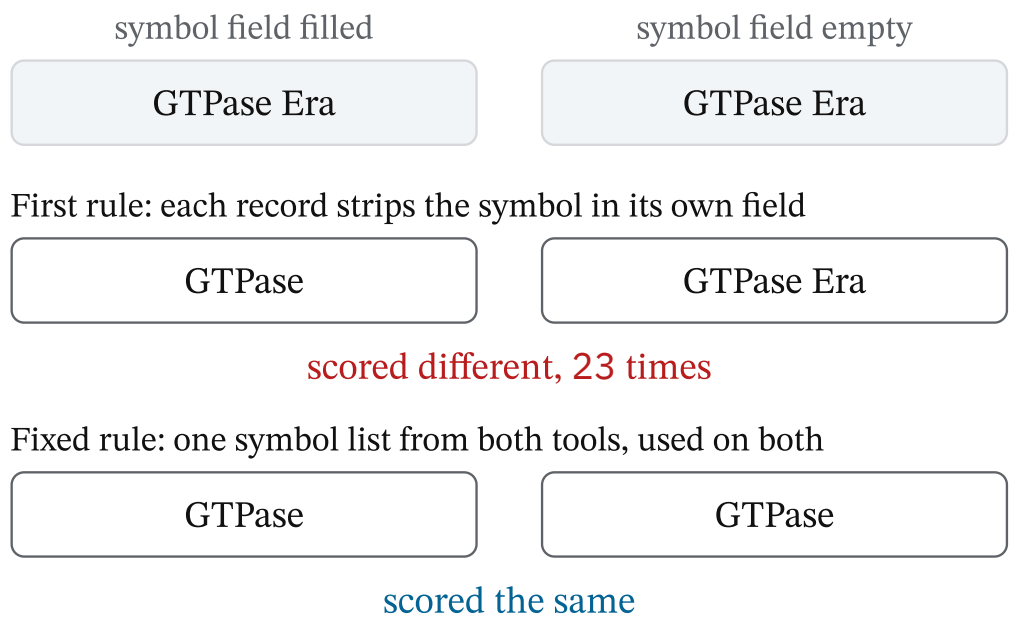


Figure 3.4 One name, GTPase Era, written the same way by both tools, where only one record has its gene-symbol field filled. Stripping each record’s own symbol shortened one side only, and the pair was scored as different 23 times; one symbol list, built from both tools and used on both sides, shortens both, and the names are scored the same.

The fix is one line of thinking rather than one line of code. Build the symbol list once per pair, from both tools, and apply it to both sides. Listing 3.1 is the comparison as it now stands.

Listing 3.1 Comparing two product names with one symbol list built for the pair, from `scripts/lib_names.py`.

```
def compare(product_bakta, gene_bakta, product_prokka, gene_prokka,
            level="primary"):
    """-> (agree, normalised_bakta, normalised_prokka)."""
    symbols = (frozenset() if level == "strict"
               else symbol_candidates(product_bakta, gene_bakta,
                                     product_prokka, gene_prokka))
    a = normalise(product_bakta, symbols, level)
    b = normalise(product_prokka, symbols, level)
    return a == b, a, b
```

Then I added an assertion, because a fix you cannot verify is a hope. If two raw strings are identical, they can never be scored as a disagreement. The pipeline checks this on every run and refuses to write the file if it fails. Across all 87,859 regions both tools called, 29,808 pairs have identical raw strings. There are zero violations.

Note the direction of the bug. It inflated the disagreement rate. It made my result more interesting.

3.7 HALF THE NAMES

Bakta and Prokka read the same 50,210 genes. They give different names to 25,977 of them. That is 51.7%, about half. These are the same regions, with both tools confident enough to write a name, and the names do not match.

The obvious objection is that this is a formatting artefact, and the check fixed in advance answers it. Compare the strings as written, ignoring only capitals, spacing and a full stop at the end, and 56.0% disagree. Compare them under my normalisation rule and 51.7% disagree. Compare them under the loosest rule I was willing to defend, dropping generic words and ignoring word order, and 50.6% disagree. Five points across the entire range from strictest to loosest. The disagreement is not punctuation.

A second objection is worth answering. Bakta has a naming style that guarantees a mismatch. When it can only match a structural domain, it writes a name ending in domain-containing protein. Prokka does not do that, so those are near-certain disagreements. There are 4,210 such regions, and 284 more where Bakta calls the protein uncharacterised and adds only the name of its open reading frame, which Prokka never matches either. Together that is 4,494 regions, 9% of the set, disagreeing almost every time. Take them out and the remaining 45,716 regions disagree 47.0% of the time.

The shorthand disagrees too. Where both tools assign a gene symbol, the symbols differ 41.1% of the time. Where both assign an enzyme number, the numbers differ 16.7% of the time.

Pangenome analyses group genes by name. Functional enrichment counts categories built from names. Resistance reports quote a gene product. Every one of those inherits whichever tool was run. None of this is a claim that either tool is wrong. I have no way to know which name is right, and neither does anybody working from annotation output alone.

3.8 A MODEL THAT READS NOTHING

Can the naming disagreement be predicted? The first answer was a warning. A model that reads nothing won.

It uses no sequence, no database matches and no information at all. It always guesses the same answer, and it got the highest score in the whole comparison. Here is how a model that knows nothing beats models that know something. Half the regions in this data are disagreements between the two tools, so a model that always says yes, this is a disagreement, is right about half the time by luck alone. The score it won on was F1, which Chapter 2 also used. It rewards catching every true case, and it gives no credit for a region correctly left alone. Say yes to everything and you catch every true case. So the model that never looks at the data scored 0.664 on F1, higher than any model that actually learned something.

This had happened before, in Chapters 1 and 2. A score means nothing until you know what a model that knows nothing would score.

Matthews correlation coefficient (MCC). A score that punishes false alarms as much as it rewards true catches, and that also gives credit for a region correctly left alone. Zero means no better than a coin flip. One means perfect.

So the honest comparison uses a different measure, MCC, with every model and baseline scored on the same regions, all from five genomes held out of training. The model that always says yes scores 0.000, as any model that gives the same answer every time must. A model that looks only at how long the protein is scores 0.028. One that looks only at whether Prokka found a similar protein in UniProtKB, a large public protein database, scores 0.141. It stands in for the baseline I had planned, which asked whether either tool had left the protein unnamed: every region here was named by both tools, so that question has the same answer everywhere. A single decision tree scores

0.261. The random forest scores 0.303. Gradient boosting, a more advanced technique, scores 0.288. The most elaborate model on that list did not win; the random forest did (Figure 3.5).

The model that always says yes

| 0.000 but 0.664 with F1, the highest F1 of all

Protein length only

0.028

Prokka's UniProtKB match only

0.141

One decision tree

0.261

Random forest

0.303

Gradient boosting

0.288

0 0.1 0.2 0.3 0.4

MCC, every model scored on the same held-out regions

Figure 3.5 MCC for the three baselines, in grey, and the three models, all scored on the same held-out regions, on an axis from zero. The model that always says yes scores 0.000, although its F1 of 0.664 was the highest of all; the random forest is highest at 0.303.

What does the random forest get right and wrong? It was tested on 12,283 regions it had never seen, and it flagged 6,105 of them as disagreements. 3,965 of those flags were correct. 2,140 were not. So for every three regions it flags, one is a false alarm, against one in two if it flagged at random. That is real signal. It is nowhere near reliable enough to trust on its own.

3.9 WHAT THE FOREST WAS READING

I ran the same random forest three times, with the same folds, the same random seed and the same way of choosing its settings. Only the columns it was allowed to see changed (Figure 3.6). The seventeen columns that describe the raw DNA include the GC content of the gene and of the hundred letters on either side, how repetitive the sequence is, how often stop codons fall in its emptiest reading frame, and three properties of the whole genome. Thirty-five come from the gene caller: the boundaries it chose, the reading frame and strand they imply, and everything computed from the protein those boundaries translate into, such as its length and the share of each amino acid. The other nine describe what each tool found in its databases.

With all 61 features, the forest scores 0.303 on MCC on the five held-out genomes. Take away everything that comes from the gene caller itself, leaving 26 features, and it scores 0.259. It barely moved. Take away the database features too, leaving the 17 that describe the raw DNA, and it scores 0.142.

That last number is generous. A test on five genomes is small, and its score moves with which five genomes happen to be in it. The steadier measure comes from the twenty training genomes: split them into five groups, hold each group out in turn while the forest trains on the other four, and average the five scores. On that measure the DNA-only forest scores 0.070, which is close to nothing. That is the honest answer to the question this chapter actually asked. Does the DNA sequence predict disagreement? Barely.

- MCC on the five held-out genomes
- MCC in cross-validation on the twenty training genomes

All 61 features



Gene-caller features taken away: 26 left



Database features taken away too: 17 left, all raw DNA



Figure 3.6 The same random forest given fewer columns, scored on the five held-out genomes, on an axis from zero: 0.303 with all 61 features, 0.259 with the gene-caller features taken away and 0.142 with the database features taken away too, leaving the 17 that describe the raw DNA. The second bar for DNA alone, 0.070, is the same forest in cross-validation on the twenty training genomes.

So which features had been carrying the score? There is a way to check this honestly. Shuffle one column, so that it no longer lines up with the right answers, and score again to see how much was lost. Do it on genomes the model never saw, so the answer is about generalisation. Then add up the damage by category. Shuffling the nine features that describe database cross-references cost 0.165. Shuffling the thirty-five features that describe the gene call itself cost 0.019. Shuffling the seventeen features that describe raw DNA cost negative 0.001. Negative means that shuffling those columns made the model slightly better. That is what noise looks like when you measure it honestly.

The top three features were how many cross-references Bakta attached, whether Prokka assigned an enzyme number, and whether Prokka matched a protein motif.

The obvious reading is that the names disagree where the evidence runs thin: neither database knows the protein well, each tool guesses, and the guesses differ. I tested that rather than assuming it, and it holds only in part. It fits the 4,494 regions where Bakta

fell back on a domain name or an open reading frame name, which disagree almost every time. Among the other 45,716, it holds on Prokka's side. Where Prokka assigned no enzyme number, the names disagree 54.0% of the time, against 42.1% where it did.

On Bakta's side it runs the other way. Where Bakta attached two cross-references or fewer, as it does about nine times in ten, the names disagree 44.2% of the time. Where it attached three, they disagree 79.8% of the time, and at four or more, 62.8%. A third cross-reference is usually an enzyme number or a match in a more specialised database, and it comes with a more specific name from Bakta, which is then more likely to differ from Prokka's more general one. That is why the number of cross-references Bakta attached is the forest's strongest feature: on Bakta's side, more evidence goes with more disagreement. Either way, what the model learned is a fact about two reference databases and how each tool names from them. It has nothing to do with the DNA sequence itself.

One decision made this visible. Every feature was given flags at creation saying where it came from, and the check reads those flags automatically rather than from a list I typed by hand. Listing 3.2 shows how the database features were declared.

Listing 3.2 The features declared as database-derived, each with two flags (derived from the gene caller, derived from a database) and a description, from `scripts/14_content_features.py`. One of the ten, `bakta_has_pfam`, never varied among the regions both tools named and was left out, which leaves the nine the model used.

```
"bakta_has_uniref": (
    0, 1, "Bakta's Dbxref contains a UniRef cluster"),
"bakta_has_pfam": (
    0, 1, "Bakta's Dbxref contains a Pfam accession"),
"bakta_has_ec": (
    0, 1, "Bakta's Dbxref contains an EC number"),
"bakta_has_is": (
    0, 1, "Bakta's Dbxref contains an ISFinder insertion-sequence
match"),
"bakta_n_dbxref": (
    0, 1, "number of cross-references Bakta attached"),
"prokka_has_similarity_hit": (
    0, 1, "Prokka's inference cites a UniProtKB protein similarity"),
"prokka_has_protein_motif": (
    0, 1, "Prokka's inference cites an HMM protein motif"),
"prokka_has_cog": (
    0, 1, "Prokka assigned a COG"),
"prokka_has_ec": (
    0, 1, "Prokka assigned an EC number"),
"mobile_neighbours_5kb": (
    0, 1, f"Bakta CDS within {MOBILE_WINDOW} bp carrying an ISFinder "
        "match; the only mobile-element context derivable from what "
        "is already annotated"),
```

Without those flags, the 26 features left once the gene-caller columns were gone would have passed for features of the DNA, and the score of 0.259 would have looked like real biology. It was not.

3.10 WHAT THIS CHAPTER GOT WRONG

I built an experiment around a disagreement that turned out to be 72 rows. The reason was sitting in the tools' own documentation, and I had not read it first.

Bakta ran on db-light, the smaller of its two reference databases, because the full one would not fit on my machine. I predicted that the smaller database would leave Bakta naming fewer genes than Prokka. I had it backwards. Of the 87,859 regions both tools called, Bakta left 12.5% with a placeholder such as hypothetical protein instead of a name. Prokka left 40.2%.

My own code counted identical text as a disagreement, 23 separate times, and that mistake made my result look better than it was.

One feature in the first experiment, on where genes are, never changed value across its 87,960 rows, so it could tell the model nothing. Nobody caught it until the second experiment, on names.

3.11 FACT OR CALL

The chapter asked whether a gene is a fact or a call, and the answer split in two.

Where does a gene start? Bakta called 87,960 genes, and the two tools disagreed about only 72 of them. That looks remarkable until you see why. Both tools find genes with the same underlying program, so this was never two independent opinions. It was one opinion, copied. Where a bacterial gene starts looks like a fact, but it is one decision, made once and inherited by everyone who has used the same software since. A fact that spread is still a copy.

What is that gene, once you have found it? Here the story turns over. For the same 50,210 genes, both tools were confident enough to give a name, and the names disagree 51.7% of the time. Without one tool's habit of naming genes after a generic protein domain, or after an open reading frame alone, it is still 47.0%. A model can predict which genes will disagree, but not from the DNA. Give the model only the features that describe the DNA, and the score falls to 0.142 on the five held-out genomes, and to 0.070 in cross-validation on the twenty training genomes. What carries the prediction is what each tool found in its reference library. Where Prokka found less, the names disagree more often. Where Bakta found more, it gives a more specific name, which is more likely to differ from Prokka's. The DNA never changed. The books each tool checked did.

So a gene is both. Where it starts is a call so old and so widely copied that it behaves like a fact. What it does is a call made fresh, by whoever has the newer library, and it disagrees over half the time. Every number from the two experiments, right and wrong, is in the repository. Chapter 4 asks a harder kind of question, not whether something happens but when, and some of its patients are still alive when the study ends.

This chapter is rewritten from the posts for Days 26 to 39 of the series.

RUN IT

Repository github.com/Qasim-Hussain-Code/bakta_prokka_disagreement_prediction

Commit [4f797c3fa09d](https://github.com/Qasim-Hussain-Code/bakta_prokka_disagreement_prediction/commit/4f797c3fa09d)

CITE THIS CHAPTER

Hussain, Q. (2026). Is a gene a fact or a call? In *Machine Learning for Biology*, chapter 3.

WHAT HAPPENS TO THE PATIENTS WHO ARE STILL ALIVE?

Model: Gradient boosting

How do you model when something happens to a patient, if for many patients it has not happened yet?

SUMMARY

Gradient boosting with a Cox partial likelihood objective, trained to rank colorectal cancer patients by their risk of recurrence or death in a cohort of 586, treating the 401 with no dated event as not yet rather than never. On 176 patients held out from training, 56 of them with an event, it reached a concordance index of 0.7289 against 0.6743 for stage alone. With both models refitted on colon cases alone, the gap narrowed to 0.7228 against 0.706 and could not be told apart from chance; adding tumour mutation count, tested on the 159 held-out patients who had it, changed the concordance index by less than a hundredth. The model ranks patients somewhat better than stage does, and it was built to rank them, not to tell any one patient how long they have.

A patient enrolls in a cancer study. Years pass. The study closes on a fixed date, the same date for everyone, and it does not matter where each patient's disease happens to be on that day. Some patients have had their event by then: a recurrence, confirmed and dated. Others have not, and are alive and disease-free when the clock stops. For them the true answer is not yet.

Every chapter so far has asked a yes-or-no question: resistant or not, bound or not, named the same or not. This chapter asks a harder kind of question. It asks when something happens. The data is cancer survival, with real patients, real follow-up time and real recurrence, and it brings a new way for the label itself to lie.

Suppose the patients still alive are treated as a no, the same as someone followed for ten years who never recurred. Then the model has been told something false. A patient followed for eight months with no event yet and a patient followed for eight years with

no event ever look identical in that label. Those are not the same fact. One is early. One is settled.

Censoring. A patient is censored when the study stops watching before the event being watched for has happened. All that is known is that the event had not occurred by the last date anyone checked.

Censoring is not unique to cancer data. Any study with a fixed end date and an event that might happen later has it: component failure, customer churn, reoffending after release. Biology just makes the stakes obvious.

There is a common wrong move here, and I named it before I could be tempted by it. Drop everyone still alive at the end, and keep only the patients with a known event. That feels safe. In fact it deletes exactly the patients who are doing best, because good outcomes take longer to prove than bad ones. The remaining data is quietly enriched for recurrence, and the model learns a population that does not exist.

So the label for this chapter is two numbers together: how long a patient was followed, and whether the thing being watched for happened before the watching stopped. The patients still alive at the end are not missing data, and they are not a problem to clean up before modelling starts. They are the central fact of the chapter. Model around them carelessly and every number that follows is wrong.

4.1 WHY GRADIENT BOOSTING, AND WHY IT NEEDS A DIFFERENT OBJECTIVE

There is a reason Chapter 3 ended on random forests. Averaging many trees is one way to reduce error. Building trees one at a time, each one correcting what the last one got wrong, is another. That second way is gradient boosting, the next model on the list set out in Chapter 2. It had already made a quiet appearance in Chapter 3, as a comparison.

Every model before this one was trained to predict one number and checked against one answer. This one predicts a number and is checked against two things at once: whether the event happened, and how long it took. Ordinary gradient boosting cannot do that. The measure a model is trained to improve is called its objective function, and it sets what counts as a good prediction. Without an objective function that understands censoring, gradient boosting does exactly what I warned against above, and scores a patient still doing well at eight months the same as one confirmed disease-free at eight years.

The fix is called the Cox partial likelihood, and the idea behind it is older than gradient boosting itself. Do not ask the model to predict a survival time directly. Ask it for a risk score, and judge that score by how well it puts patients in the order in which their events came.

In practice it works like this (Figure 4.1). At the moment each event occurs, look at everyone still at risk, meaning everyone who has not yet had the event and is still being followed. The patient it happened to should have scored higher risk than everyone else still in the running at that moment. Do that at every event in the data and add it up, and the result is a single number the model can be trained to improve. A censored patient takes part in every one of these comparisons up to the last date anyone checked on them, and in none after it. The model is never asked to know exactly when. It is asked to know who is more at risk than whom, at every moment someone's risk became real.

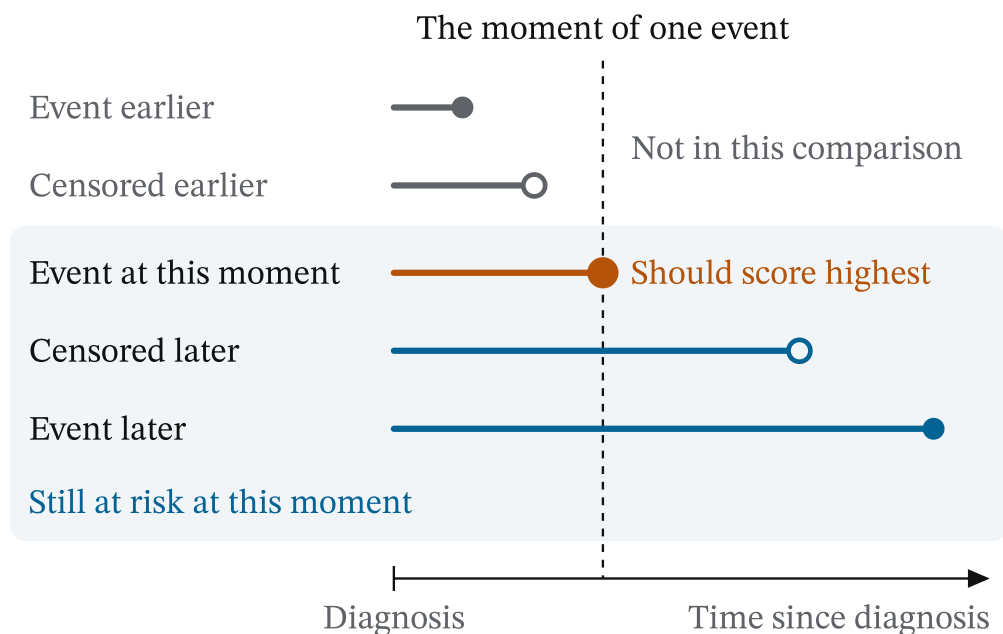


Figure 4.1 The comparison the Cox partial likelihood makes at one event, sketched for five patients followed from diagnosis to an event (filled dot) or to a last check with no event (open circle). The patient with the event should have a higher risk score than everyone else still at risk at that moment (shaded); patients whose event or last check came earlier take no part in it, and the same comparison is made at every event.

The two numbers of the label reach the model as one. Listing 4.1 shows how: the follow-up time is kept positive for a patient who had an event and made negative for one who was censored.

Listing 4.1 The label handed to the boosted model, from `scripts/survival_utils.py`.

```
def cox_labels(time, event):
    """XGBoost survival:cox labels: positive time for events, negative for
    censored."""
    time = np.asarray(time, dtype=float)
    event = np.asarray(event).astype(bool)
    return np.where(event, time, -time)
```

One consequence was worth sitting with before any result existed. A model can have excellent concordance, putting patients in the right order of risk, and still be useless for telling one specific patient how long they have. Ranking who is at higher risk is a different skill from predicting a date. This chapter is about the first skill, and I named that limit before a number could make it inconvenient to say.

4.2 THE RULES

I wrote the rules down before I had loaded a single row.

RULES FOR THIS CHAPTER

Event.

The first confirmed recurrence, or death from any cause, whichever comes first. Time is measured from diagnosis.

Censoring.

Anyone with no confirmed event by their last recorded contact is censored at that date. Patients still in active follow-up and patients lost to follow-up are treated the same: alive and event-free, as of the last date anyone checked.

Exclusion.

Patients with less than thirty days of follow-up are excluded before the split, not after.

Split.

Grouped, as in Chapter 3. One patient, one group, never divided between train and test.

Metric.

Concordance index, not accuracy and not F1.

Baseline.

Stage alone, on the same patients.

The label is disease-free survival, not overall survival and not progression-free survival on treatment. Death from any cause is in there deliberately. A patient who dies in a car accident before their cancer could recur is a hard case, and different studies handle it differently. I counted it as an event, not a censor. Pretending that patient's follow-up simply continued would be its own quiet lie. I recorded the choice in advance so that I could not change it later if the number looked better the other way.

Starting the clock at diagnosis was a choice too. Enrolment and the end of primary treatment would each give a different clock, and three different clocks make three different chapters.

Listing 4.2 is the event rule as the pipeline applies it to each patient. The earlier of a dated recurrence and a dated death becomes the event. A patient with neither is censored at the latest recorded contact.

Listing 4.2 Setting each patient's time and event, from `scripts/02_build_cohort.py`.

```
candidates = [d for d in (first_recurrence, first_death) if d is not None]
if candidates:
    time, event = min(candidates), 1
    if first_recurrence is not None and first_recurrence == time:
        event_type = "recurrence"
    else:
        event_type = "death"
else:
    contacts = [c for c in ([number(row.get("last_contact_days_to"))
                           + fu_contact[barcode] + fu_death[barcode])
                           if c is not None])
    if not contacts:
        audit["dropped_no_time"] += 1
        continue
    time, event, event_type = max(contacts), 0, "censored"
```

The thirty-day threshold exists because so short a follow-up says nothing, and including those patients would let the model learn from noise dressed up as signal. The grouped split did not need deciding, only restating.

Concordance index (C-index). Of two randomly chosen patients whose order is known, does the model rank them in the correct order by risk? The order is known only when the patient followed for less time had the event, so pairs in which that patient was censored are left out. Chance is 0.5. A perfect ranking is 1.0.

The concordance index asks a narrower question than accuracy does. The model learns to get orderings right, and it is scored the same way.

The baseline is stage alone: whatever a clinician already knows from TNM staging, and nothing else. TNM records how far the primary tumour has grown (T), whether the cancer has reached nearby lymph nodes (N) and whether it has spread to distant parts of the body (M), and the three are combined into a single stage. If gradient boosting could not beat stage, the honest chapter would be the one that said so.

4.3 FOUR WAYS THIS COULD BE WRONG

I did not know whether this chapter would work, so I wrote down why it might not before I found out (Figure 4.2).

- 1 The cohort is not a random sample of cancer patients
Check: none possible from within the cohort

- 2 One clinical fact may count as several features
Check: rank correlation among stage, pT, pN and others

- 3 Events, not patients, limit a survival model
Check: below 50 events, a result carries a warning

- 4 Closer follow-up, not the tumour, may predict outcome
Check: early follow-up contacts against later events

Figure 4.2 The four ways the chapter could be wrong, named before the pipeline ran, and how each could be checked.

The first is the cohort. Public cancer cohorts are not a random sample of cancer patients. They are the patients whose data made it into a study, at institutions with the funding to run one, filtered by inclusion criteria that already excluded the sickest and most complicated cases. Whatever this model learns, it learned from that population and not from cancer patients generally.

The second is structural rather than a matter of sampling. Stage, grade and tumour size are correlated with each other. Bigger tumours tend to be later stage, and later stage tends to mean higher grade. A model can look as if it found three independent signals when it found one signal, badly, three times. Chapter 3 met a form of this, which I call

circularity: a model detecting the tool that made the label rather than the biology underneath it. The version here is one clinical fact, detected through three correlated proxies and reported as three important features instead of one.

This data turned out to record neither grade nor tumour size, so the check for this warning ran on stage and the other descriptors that were recorded, with pT, the T category of TNM, standing in for tumour size, and pN, the N category, for the lymph nodes. Stage is built partly from those two, so the check still asked the same question.

The third is a small event count. Survival models are limited by how many events occurred, not by how many patients were enrolled. A cohort of a thousand patients with sixty events behaves, for fitting purposes, much closer to a cohort of sixty. If confirmed recurrences in this cohort number in the low hundreds, the honest ceiling is lower than the sample size suggests.

The fourth is the one I was most likely to get away with unless I checked for it directly. How closely a patient is followed can itself carry information that has nothing to do with biology. Some patients are followed more closely because a clinician was already worried. That worry, not the tumour, can end up predicting outcome.

None of these breaks the chapter. Each one changes what a result would mean. Each one would be checked and reported, whether or not it changed the headline number.

4.4 WHAT HAPPENED

The cohort is 586 patients from two public colorectal cancer datasets. Of these, 185 had an event: the cancer came back, or the patient died. The other 401 had no event with a date, as of the last time anyone checked on them. One of them did have a recurrence on record, but without a date it could not start a clock, so that patient was censored with the others.

The model was given ten pieces of information about each patient, things like cancer stage, lymph node status and treatment history.

I tested it on 176 patients the model had never seen, 56 of whom had an event. The model scored 0.7289 on the concordance index. It got the ordering of a pair of patients right about seventy-three times out of a hundred. To score stage alone on the same patients, I fitted a Cox proportional hazards model, a regression model built for survival data, to stage and nothing else on the 410 training patients, giving patients whose stage was not recorded a level of their own. Stage alone scored 0.6743.

Besides the four warnings, I had committed to two checks before I knew what they would show.

The first asked what happens when only colon cancer is considered, leaving out rectal cancer. I refitted both models on the 297 colon patients in the training set, 96 of whom had an event, and scored them on the 128 colon patients held out, 42 of whom had one. The model scored 0.7228 against 0.706 for stage. That is still an improvement, but a smaller one, and this time the gap is not clearly bigger than what chance alone could produce. Both groups agree that the model does better. Only the larger, mixed group is confident enough to say that the improvement is real rather than noise.

The second asked whether knowing how many mutations a tumour carries adds anything. Of the 586 patients, 540 had this data. Their mutation counts split into two groups, as expected in this cancer: 77 of the 540 tumours, 14.3%, carry more than 10 mutations per megabase, which is the hypermutated fraction already known in colorectal cancer. So I trust the measurement. I refitted the model with and without mutation count on the patients who had it, and scored both on the 159 of them held out, 52 with an event. Without mutation count the model scored 0.7243, and with it 0.7302. The difference, 0.0059, is less than a hundredth, and its 95% interval runs from negative 0.0055 to 0.0186. That takes in zero, so the change could easily be nothing at all. There is no evidence here that mutation count helped this model predict risk.

Then the four warnings. The first, that this cohort is not a random sample of cancer patients, cannot be tested from within the cohort. Every number in this chapter describes these patients, not colorectal cancer patients in general.

The second warning asked whether the model would find one real signal and report it as two or three separate important features, making it look as if it used more information than it did. That one did happen. The two most important features were stage and pN, the lymph node category, and the two move together. Across the 566 patients with both recorded, their rank correlation is 0.8289, where 1 would mean that they always rise together. They are largely one fact about the patient, counted twice.

The third was the event count. Before fitting, I had fixed a floor of 50 events, below which a result would carry a warning. The main split, colon and rectal cases together, cleared it, with 129 events among the training patients and 56 among those held out. The colon-only test set did not. It had 42, and at that number of events the colon gap cannot be told apart from chance.

The fourth asked whether being checked on more often, because a doctor was more concerned, itself went with a worse outcome, so that the model could have learned the doctor's worry rather than anything about the cancer. I found no evidence that it did. The test I had fixed before looking at the data counted each patient's follow-up contacts in the first 90 days after diagnosis, and asked whether that count predicted an event after those 90 days. It could not run. Follow-up forms in these datasets are filed once a year, so only 4 of the 544 patients still at risk at 90 days had any contact inside the window. After seeing that, and before testing anything against outcome, I widened the window to the first 365 days. Each extra contact in that year went with a hazard ratio of 0.455 for an event afterwards. A hazard ratio of 1 would mean no effect at all, and the 95% interval around 0.455, from 0.1678 to 1.2341, takes in 1.

4.5 WHAT THE RESULT DOES AND DOES NOT SHOW

The model ranks patients by their risk of recurrence or death somewhat better than stage alone does. That is the finding.

The improvement over stage holds up only in the larger, mixed group. On colon cases alone it points the same way but cannot be told apart from noise. The second warning came true, so the model's list of important features overstates how many separate signals it used. And the result says nothing about how long any particular patient has.

Four chapters have now used four models. Logistic regression drew a straight line through a decision. A single decision tree could be read out loud, one split at a time. A random forest let many trees vote together, each one blind to what the others saw. Gradient boosting built trees one after another, each correcting what the last got wrong, and here it was adapted to patients whose outcome had not happened yet. That is the standard path through tree-based models, the one most textbooks follow in roughly the same order, and it ends here. Chapter 5 sets out to use a support vector machine, the next step on that same standard progression.

This chapter is rewritten from the posts for Days 40 to 46 of the series.

RUN IT

Repository github.com/Qasim-Hussain-Code/cancer_survival_ml_pipeline
Commit [85d831ca383c](https://github.com/Qasim-Hussain-Code/cancer_survival_ml_pipeline/commit/85d831ca383c)

CITE THIS CHAPTER

Hussain, Q. (2026). What happens to the patients who are still alive? In *Machine Learning for Biology*, chapter 4.

GUT BACTERIA AND CARDIOVASCULAR RISK

Model: Logistic regression (support vector machine planned)

Can the presence of the bacterial machinery that starts the production of TMAO stand in for measuring TMAO at all?

SUMMARY

The chapter set out to ask whether a support vector machine, shown only the abundance of the bacterial genes that start the production of TMAO, could predict atherosclerotic cardiovascular disease, a design that needs shotgun metagenomics because the key gene, *cutC*, moves between bacteria by horizontal transfer. The public data could not support that design, and the support vector machine never ran; the question then became whether TMAO helps predict cardiovascular risk on top of everything doctors already check, and the data to answer that directly is not public either. In a narrower public dataset of 750 patients, followed for nine months after a heart procedure to see who had chest pain again, adding TMAO to a logistic regression on its three precursors raised the area under the ROC curve, scored on patients each model had not been trained on, from 0.848 to 0.873, a difference of 0.0256 with an interval of 0.0075 to 0.0436, and the rise met both parts of the rule fixed before any model was fitted. A penalised logistic regression on the whole panel came back nearly perfect, with an area of 1.0; TMAO ranked around the 64th percentile of the substances measured in every patient. The deposit numbers every patient who had chest pain again before every patient who did not. A check fixed in advance found structure along that numbering within both groups, and without the lab processing order, the dataset cannot fully rule out a handling effect, so the rise cannot be read as strong evidence about TMAO specifically.

Somewhere in your gut, bacteria are doing something no cell in your body can do.

Your own enzymes cannot break the bond between carbon and nitrogen in a choline molecule. Certain gut bacteria carry one that can, a glycyl radical enzyme called TMA-lyase, and they use it constantly. They cleave the choline and carnitine that come from the steak or eggs you ate, and release a waste gas called trimethylamine, or TMA. The gas crosses into your bloodstream and reaches your liver, where a single enzyme, FMO3, oxidises it into trimethylamine *N*-oxide. TMAO.

From there it does real damage. It pushes macrophages to swallow more cholesterol than they should, turning them into the foam cells that build atherosclerotic plaque. It blocks the pathway that clears cholesterol back out of your arteries. It makes platelets clot more readily than they should. Elevated TMAO is now one of the more consistently replicated predictors of cardiovascular risk in the literature, and it predicts risk independently of cholesterol itself.

That gives this chapter its question. The enzyme is bacterial, and the bacteria that carry it can be identified. If a support vector machine is shown only the abundance of those TMA-producing bacteria, with no host genetics and no cholesterol panel, can it predict cardiovascular risk from that alone?

Measuring TMAO itself needs a specialised laboratory assay, and almost nobody who could benefit from knowing their risk will ever have it run. The question underneath the question is whether the presence of the machinery that makes TMAO can stand in for measuring TMAO at all.

One limitation, named now rather than discovered later. TMA-lyase only matters if there is substrate to cut. A person can carry every one of these bacteria and still produce almost no TMAO, if choline and carnitine barely appear in what they eat. Without diet data, a model of this kind has a hard ceiling on what it can know, and that ceiling is part of the chapter, not a footnote added after the fact.

5.1 THE ENZYME IS NOT THE ORGANISM

The plan as I first stated it was to give the model the abundance of bacteria known to carry TMA-lyase. That sentence hides a problem, and the problem decided what data the chapter would need.

The gene for choline TMA-lyase is called *cutC*, and it does not follow the family tree. Two genera, *Desulfosporosinus* and *Proteus*, carry it in every sequenced genome. Almost nothing else does. Ninety-eight of 2,719 sequenced *Escherichia coli* genomes carry a *cutC* homologue, four per cent. The other ninety-six per cent do not. Same genus, same name in a results table, and only one in twenty-five can perform the reaction.

Genus and homologue. A genus, plural genera, is a group of closely related species, and it is the level at which 16S sequencing usually reports what is in a sample. A homologue is a recognisable version of the same gene found in another organism.

The gene spread by horizontal transfer, passed sideways between organisms rather than inherited down a lineage. It turns up in patches across four bacterial phyla, and it is absent from most members of most of the genera that carry it. So the obvious approach does not work.

Most public microbiome data is 16S sequencing. It reads one marker gene and reports which genera are present, and in what proportion. It does not report which genes those organisms carry. If I hand a model the abundance of a genus that sometimes carries *cutC*, I am not handing it the pathway, the genes that turn choline and carnitine into TMA. I am handing it a proxy for a proxy, and how good that proxy is depends on what fraction of the genus carries the gene in that particular person’s gut. 16S sequencing cannot measure that fraction.

Shotgun metagenomics can. It sequences everything, so *cutC* can be counted directly as a gene, whichever organism happens to carry it (Figure 5.1). The two methods are different experiments with different ceilings, and choosing between them was the first decision the chapter had to make in public.

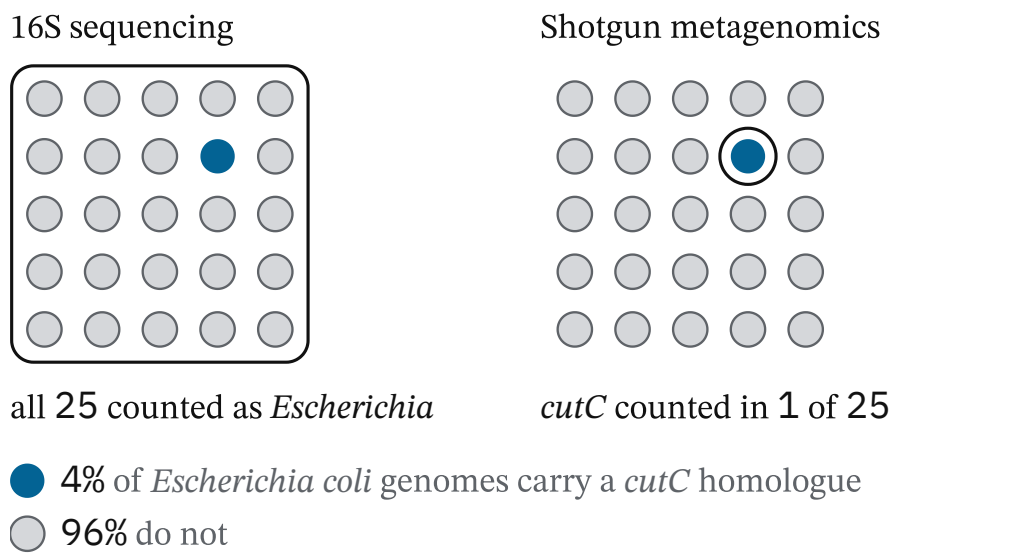


Figure 5.1 Twenty-five *Escherichia coli* genomes, read two ways. One of them carries a *cutC* homologue, matching the four per cent of *Escherichia coli* genomes that do. 16S sequencing counts all twenty-five as the genus *Escherichia*, whether or not they carry the gene; shotgun metagenomics counts *cutC* itself, and finds it in one.

5.2 WHY A SUPPORT VECTOR MACHINE WAS PLANNED

The metagenomic design was planned around a support vector machine. Its features, the measurements the model would be given for each person, were a short list: the six genes of the pathway that makes TMA, each with a mechanistic reason to be there. Six numbers per person, a few hundred people, and a boundary that might curve rather than split cleanly. That is close to the exact problem support vector machines were built to solve.

The models of Chapters 2 to 4 all worked by cutting the data into groups. Is this gene above a certain level, yes or no? Then the model asks again. Stack enough of those cuts together and you get a decision tree, or a forest of them, or trees that correct each other. All of it is still cutting.

A support vector machine does something different. It measures the gap. Picture each sample as a point, placed according to its six gene abundances. The model looks at the healthy samples and the sick samples, finds the widest possible empty space between them, and draws its boundary straight down the middle of that space (Figure 5.2).

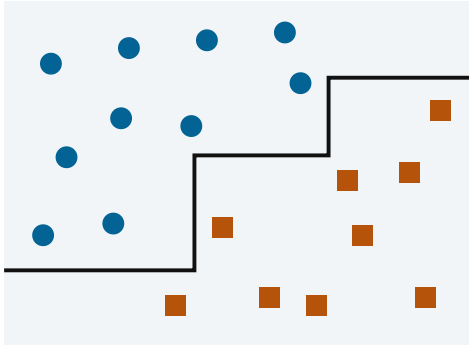
The width is the point. Most models are content with any boundary that separates the training data correctly. There are usually many such boundaries, and on the data already in hand they all look equally good. This model picks the one with the most room on either side, because room is what helps when the next patient does not look exactly like anyone it has seen before.

Something else follows from this. Only the patients sitting closest to the boundary shape it. Everyone comfortably inside their own group could be removed from the data, and the boundary would not move. On a small group of patients, that is useful, because the model is defined by its hardest, most borderline cases and not by the easy majority. Those borderline patients are called support vectors, a vector being one patient's list of measurements, and they give the model its name.

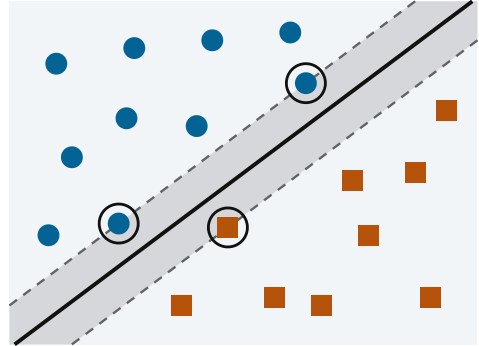
Sometimes no straight line can separate two groups, however it is angled. For that case the model can change how it measures distance between patients, a rule called the kernel. Changing that rule amounts to stretching the space until a straight line does work. Nothing new is measured. Only the notion of near and far changes, and a straight boundary in the stretched space comes back curved when it is drawn on the original data.

Whether the straight version or the curved version would work better here was something to test, not assume, and it was to be decided using only the training data, before any final test.

A decision tree
one threshold, then another



A support vector machine
a line down the widest gap



● healthy ■ sick ○ support vector

Figure 5.2 Two ways to draw a boundary between the same healthy and sick samples, shown as illustrative points. A tree builds its boundary from one threshold after another; a support vector machine places one straight line down the middle of the widest gap between the groups. Only the ringed points at the edges of the gap, the support vectors, decide where that line lies.

5.3 THE RULES

The rules for the metagenomic design were written before anything was downloaded. The last three items in the box belong to the narrower dataset the chapter later moved to, described below.

RULES FOR THIS CHAPTER

Fixed before any data was downloaded.

The data must be shotgun metagenomics, not 16S. If no suitable shotgun cohort with cardiovascular outcomes exists in public data, the chapter reports that and stops. It does not fall back to 16S.

Features.

Gene abundances, not taxa: *cutC* and *cutD* for choline, *cntA* and *cntB* for carnitine, and *yeaW* and *yeaX*, which do the same job by a different route. Each is normalised to total reads. Nothing else goes in: no age, no cholesterol, no blood pressure.

Outcome.

Atherosclerotic cardiovascular disease, present or absent, taken from the cohort's own clinical labels. What counts as a case is decided before the number of cases is known.

Split.

By person, not by sample. Every sample from one person goes on the same side, train or test, never both.

Metric.

Area under the ROC curve, with a confidence interval on every score.

Baseline.

All six genes added together into a single number, total pathway abundance, and classification on that alone.

Changed because the data is not public.

The question had become whether TMAO helps predict cardiovascular risk on top of everything doctors already check, and the data to answer that directly is not public. The chapter moved to a public dataset of 750 patients and asked two narrower questions of it. Both were answered with logistic regression, not a support vector machine, and every model was scored on patients it had not been trained on.

Fixed before any model was fitted.

Adding TMAO to a model of its three precursors counts as a real improvement only if two things hold. The confidence interval on the difference in area under the ROC curve excludes zero, and the decision curve of the model with TMAO, a measure of how useful its predictions would be for deciding whom to treat, lies above that of the model without it across a substantial part of the range of risk thresholds.

Looked at before these rules were written.

TMAO and its three precursors, summarised in each group of patients, with a rank-sum test on TMAO. Which way the TMAO difference ran was therefore known in advance.

The first rule follows from the way *cutC* is spread between bacteria. Genus abundance is a broken proxy for the pathway, and with it I would be measuring the wrong thing carefully. Normalising to total reads, the number of DNA fragments sequenced from the sample, stops a more deeply sequenced sample from looking like a sicker one. The clinical variables stay out because the question is what the pathway alone can carry; adding a clinical panel answers a different and much easier question.

The split is by person because some cohorts collect several stool samples from the same individual. If those samples were divided between training and testing, the model would meet a person in training, recognise them again in the test set, and score well without having learned anything about the pathway.

Area under the ROC curve. Pick one patient with disease and one without, at random. The area is the chance that the model gives the sick one the higher score. At 0.5, the model is guessing; at 1.0, it is always right.

The confidence interval is there because a number built from a few hundred people has a range around it, and hiding that range would flatter the result.

The baseline is deliberately unglamorous. If six separate gene abundances cannot beat their own sum, the useful signal is simply how much pathway is present. Which genes, and in what proportion, would be adding nothing. That would be a real result rather than a failed chapter, and it would have been the headline.

5.4 FOUR WAYS THIS COULD BE WRONG

I wrote down four ways the chapter could be wrong before I knew which of them was true.

The first is that gene presence is not gene expression. A metagenome shows that *cutC* is there. It does not show that the organism is transcribing it, that the enzyme is working, or that TMA is being produced on the day the sample was taken. Every result of this design would be about capacity, not activity. Those are not the same claim.

The second is the substrate problem from the opening, still unsolved. Without dietary data, the model has a ceiling it cannot see past. If the cohort had diet records, I would report results with and without them. If it did not, that limitation would stay in the headline.

The third is that metagenomic abundances are compositional. They are proportions and sum to a fixed total, so one gene rising forces every other gene to fall, whether or not anything real changed. Correlations in such data can be produced entirely by that constraint. I committed to reporting results on a centred log-ratio transform as well as on raw proportions, and to treating any disagreement between the two as the finding. The transform takes the logarithm of each gene's abundance and subtracts the average logarithm across all the genes in the same sample, so that each gene is read against the others in its own sample rather than as a share of a fixed total.

The fourth is geography. Gut microbiome composition varies enormously by population, diet and country, and a model trained on one cohort may be detecting where the samples came from rather than who has disease. If more than one cohort was available, I would test across them. If only one was, the result would travel no further than that population.

All four were written for the metagenomic design, and that design never ran, because the public data could not support it. The tests that did run, on the narrower dataset described next, used substances measured in plasma rather than genes counted in stool, so the first three do not apply to them. The fourth applies only in its fallback form. That dataset is a single cohort, and what it shows travels no further than that population.

5.5 A NARROWER DATASET

The question had become whether TMAO helps predict cardiovascular risk on top of everything doctors already check. The data to answer that directly is not public. The studies that hold it keep it behind institutional approval, which can take months with no guarantee of a yes, and I decided not to chase it any further. What I found instead was something smaller.

It is a public dataset of 750 patients, all of whom had recently had a heart procedure, followed for nine months to see who had chest pain again. It measured TMAO and the three things TMAO is made from, choline, betaine and carnitine, in every single patient, with no gaps (Figure 5.3). The dataset lists a panel of 600 substances, those four among them, and 405 of the 600 were measured in every patient.

This dataset cannot answer that question. It has no medical history and no other risk factors, so there is nothing for TMAO to add value on top of. The patients had already had heart problems, so it says nothing about predicting a first heart attack in a healthy person.

What it can answer is narrower. Does TMAO carry any extra information beyond the three ingredients it is built from? And, separately, does a model built on all 600 substances hold up when it is tested properly, on patients it was not trained on, or does it only look good on paper?

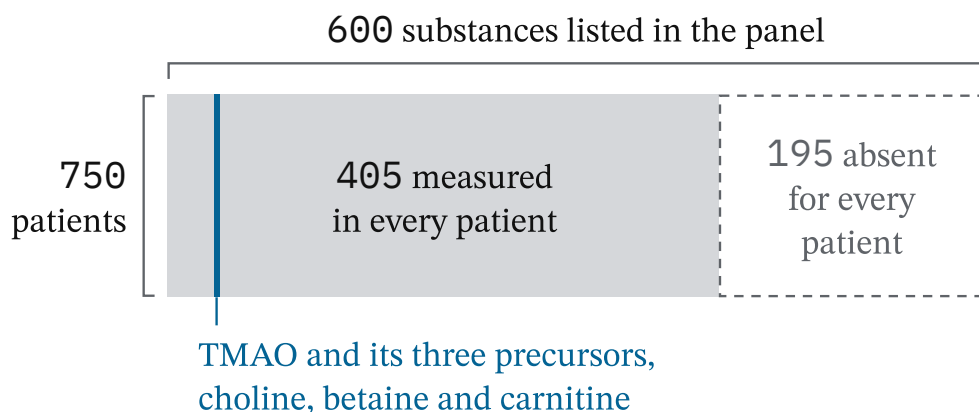


Figure 5.3 The narrower public dataset drawn as a table, with its 750 patients as rows and the 600 substances listed in its panel as columns, the widths to scale. Of the 600, 405 were measured in every patient, TMAO and its three precursors among them; the other 195 are absent for every patient.

No support vector machine ran on this dataset. I answered both questions with logistic regression, the model of Chapter 1. For the first, a logistic regression on the three ingredients was set against the same model with TMAO added, with no penalty on either. For the second, a logistic regression was given all 600 substances and penalised to keep its weights small, and the strength of that penalty was chosen inside each training set, so that the patients held back for scoring played no part in choosing it. Every model was scored on patients it had not been trained on, by cross-validation as in Chapter 2. The 750 patients were divided into five parts, each model was trained on four parts and scored on the fifth, in turn, and the whole division was made five times, with different parts each time.

I wrote down the rules for judging both answers before any model was fitted, but not before I had looked at the data. To check that the dataset was usable at all, I had already summarised TMAO and its three ingredients in each group of patients and run a rank-sum test on TMAO, so I knew which way the TMAO difference ran before the rules existed. That is a departure from an ideal pre-registration.

The rule for the first question, set out in the box earlier in the chapter, has two parts, and they test different things. An interval on the difference in area that excludes zero shows that the difference can be detected. The decision curve asks whether using the model with TMAO would lead to better decisions than using the model without it. Had the first part held and the second failed, the rise would have been reported as detectable but clinically negligible.

Decision curve. A doctor treats a patient once the predicted risk passes some threshold, and where that threshold sits says how many unnecessary treatments the doctor will accept to catch one real case. At each threshold, a model's net benefit is the share of patients it rightly flags, less the share it wrongly flags weighted by that trade. A decision curve plots net benefit across a range of thresholds.

5.6 WHAT CAME BACK

Two results came back.

The first answers the question about TMAO itself. Adding TMAO to a model of its own three building blocks, choline, betaine and carnitine, raised the area under the ROC curve for predicting who would have chest pain again from 0.848 to 0.873. More precisely, the model without TMAO scored 0.8475, with a confidence interval of 0.8132 to 0.8818, and the model with it scored 0.8731, with an interval of 0.8402 to 0.906. The difference was 0.0256, with an interval of 0.0075 to 0.0436, clear of zero. The decision curve of the model with TMAO lay above that of the model without it across 0.9821 of the range of thresholds, almost all of it. Both parts of the rule were met, so by the standard fixed before any model was fitted, that is a real improvement.

The second is the one I trust more. The model built from the whole panel, in effect the 405 substances measured in every patient, since the other 195 were empty for everyone, came back nearly perfect at separating the patients who had chest pain again from those who did not. Scored on patients it had not been trained on, its area under the ROC curve was 1.0, and its accuracy, the share of patients it put in the right group, was 0.9973. Even so, it only partly met the rule I had fixed for it in advance. Its accuracy, overall and within each group of patients, was far above the floor of 0.85 that the rule set, but its calibration slope, 2.97, lay outside the band of 0.80 to 1.25 that the rule also required. A slope that far from 1 means the predicted risks cannot be read as probabilities, even though the model ranked the patients almost perfectly.

A score that high on a hard clinical question is usually a sign to look closer rather than to celebrate. I looked closer. A large share of the substances in the panel, and not only TMAO, could each separate the two groups of patients on their own.

That changes how the first result reads. Ranked against the other 404 substances measured in every patient, TMAO comes in around the 64th percentile, behind roughly 144 others. One of those is betaine, one of its own ingredients and already part of the model that TMAO was added to. Listing 5.1 shows how that ranking is made: each substance is scored on its own by its area under the ROC curve, in whichever direction separates the two groups better, and TMAO is placed among them.

Listing 5.1 Scoring every substance measured in every patient on its own and placing TMAO among them, from `src/tmao_cvd/reanalysis.py`.

```
outcome = frame[OUTCOME].to_numpy(int)
complete = [m for m in metabolite_columns(frame) if frame[m].notna().all()]
auc = pd.Series(
    {
        m: max(roc_auc_score(outcome, frame[m]), 1 - roc_auc_score(outcome,
frame[m]))
        for m in complete
    }
)
precursor_auc = auc[list(PRECURSORS)]
best = precursor_auc.idxmax()

return PanelContext(
    marker=TMAO,
    univariate_auc=float(auc[TMAO]),
    percentile=100 * float((auc < auc[TMAO]).mean()),
    ranking_higher=int((auc > auc[TMAO]).sum()),
    panel_size=len(complete),
    best_precursor=str(best),
    best_precursor_auc=float(precursor_auc.max()),
    panel_median_auc=float(auc.median()),
)
```

5.7 WHAT THE RESULT DOES AND DOES NOT SHOW

These are results about a dataset more than about TMAO.

This is a public dataset, and like many public deposits it does not include, alongside the results, the order in which the lab processed the samples. Without that, there is no way to fully rule out that something about how the two groups were handled, rather than their biology, is doing some of the work. Instruments drift over a long run, for example, so if most samples from one group had been processed before most from the other, the drift alone could make many substances differ between the groups.

The deposit's own numbering makes that more than a hypothetical. Samples 1 to 210 are exactly the 210 patients who had chest pain again, and samples 211 to 750 exactly the 540 who did not. The plan that held the rules also held a check on this. Within each group separately, so that the difference between the groups played no part, it asked whether each substance's level rose or fell with the sample number. If the numbering had nothing to do with the measurements, about 5% of the 405 substances would appear to do so by chance. Among the patients who had chest pain again, it was 16.8%, and among the others 15.6%. That does not show the samples were run in numbered order, because the numbering need not be the order the lab used. It does mean that

every difference between the two groups in this dataset may be tangled up with that order, to a degree the deposit cannot measure. That is a limit of what this particular dataset can tell us.

Seen against that, the TMAO number needs a caveat. I report it exactly as I said in advance I would. It cannot be read as strong evidence about TMAO specifically, given what the dataset can and cannot rule out.

The question of whether TMAO helps predict heart disease risk in people who are still healthy stays open. The code, the rules written before each test, and every result in this chapter are in the repository.

That makes five chapters and five models, though the fifth was planned and never run. Every chapter so far has paired one model with one biological question chosen to make that model earn its place. Here the support vector machine, which looks for the widest possible gap between two outcomes rather than any line that happens to separate them, was paired with the metagenomic question, and both results came instead from logistic regression, the model of Chapter 1. Chapter 6 takes a question where no single line or split will do, and a model that learns for itself which positions in a peptide act together.

This chapter is rewritten from the posts for Days 47 to 54 of the series.

RUN IT

Repository github.com/Qasim-Hussain-Code/tmao_cardiovascular_risk_prediction
Commit [9985756c2db6](#)

CITE THIS CHAPTER

Hussain, Q. (2026). Gut bacteria and cardiovascular risk. In *Machine Learning for Biology*, chapter 5.

NEOANTIGEN IMMUNOGENICITY FOR CANCER VACCINE DESIGN

Model: Neural network

Given peptides already displayed on a tumour cell, can a neural network predict which ones a T cell will actually notice?

SUMMARY

A neural network was trained on peptides that all bind HLA-A*02:01, a cohort of 1,200 with 606 immunogenic and 594 not, split by sequence cluster rather than by peptide into 899 peptides for training and 301 for validation, 146 of those immunogenic. On the validation peptides the selected network scored 0.610 AUPRC and 0.665 AUC against 0.591 and 0.629 for a baseline that used binding affinity alone, with chance at 0.485 and 0.5, and that margin of 0.019 in AUPRC is the size of the network's own run-to-run variation. Weaker binders turned out to be more often immunogenic than tighter ones, with two explanations the data cannot separate. The pre-registered final test on TESLA's tumour-derived peptides never ran, because the table of validated peptides could not be retrieved: the public deposit holds only raw sequencing, and the supplement that carries the table was blocked by a browser challenge at PMC while Europe PMC refused the article as not open access.

Somewhere on the surface of a tumour cell, a protein is sitting that the immune system has never seen before. Cancer cells accumulate mutations, and some of those mutations change a protein's sequence enough that a fragment of it, displayed on the cell surface, looks foreign. That fragment is called a neoantigen, and it exists nowhere else in the body.

Chapter 2 asked whether a peptide binds MHC and gets displayed at all. That question mattered, but it was only the first hurdle. Most displayed neoantigens are still ignored. A T cell has to notice the fragment and mount a response, and most of the time none does.

MHC and HLA. Major Histocompatibility Complex class I is the molecule on a cell's surface that holds a short protein fragment in a groove and shows it to passing T cells. HLA is the human form of that molecule, and HLA-A*02:01 is one of its variants, or alleles.

Here is the question this chapter asks. Given candidate neoantigens and the MHC molecule presenting them, can a neural network predict which ones will actually provoke a T-cell response, rather than which ones merely get displayed?

The question underneath that question matters for a practical reason. A personalised cancer vaccine can only carry a small number of targets. A candidate that gets displayed but is never noticed wastes a manufacturing slot that a genuinely immunogenic one could have filled.

One limitation belongs here and not at the end. The model can learn what tends to provoke a response in general. It cannot see a particular patient's own T-cell repertoire, and two people can react differently to exactly the same neoantigen. That ceiling holds for every number this chapter reports.

6.1 WHY A NEURAL NETWORK

Immunogenicity depends on interactions across the whole peptide, position by position, combined with which HLA molecule is presenting it. That is not a boundary a single line or split can capture. It is closer to a pattern a network has to learn.

Chapter 5 took up a support vector machine, which finds the widest boundary between two outcomes. It was chosen for six numbers per person, the abundances of the genes of one bacterial pathway, and that design never ran, because the public data could not support it. The approach works when the features are few and the relationship between them is fairly stable.

This chapter hands the model the peptide itself, nine amino acids in sequence. Whether a peptide is noticed by a T cell does not depend on any one position alone. It depends on how the positions combine, an anchor residue holding the peptide in the MHC groove while a different position further along is what the T-cell receptor actually touches. Change one position and the meaning of another can change with it.

A support vector machine can be given those combinations by hand, multiplying features together before fitting the boundary. A network learns which combinations matter on its own, from the data, without being told in advance which positions

interact. That is the actual difference this chapter is testing. Not a bigger model for its own sake: a model built for inputs where position matters and positions talk to each other.

The architecture choice is not exotic. A small feedforward network reading the encoded sequence, compared against a version built explicitly to model interactions between positions, with the choice made on validation performance before the TESLA benchmark is ever opened (Figure 6.1).

Feedforward network. A model that passes numbers forward through a few layers of weights to a single prediction, with nothing looping back. It takes numbers rather than letters, so each of the nine amino-acid positions has to be encoded before the network sees it.

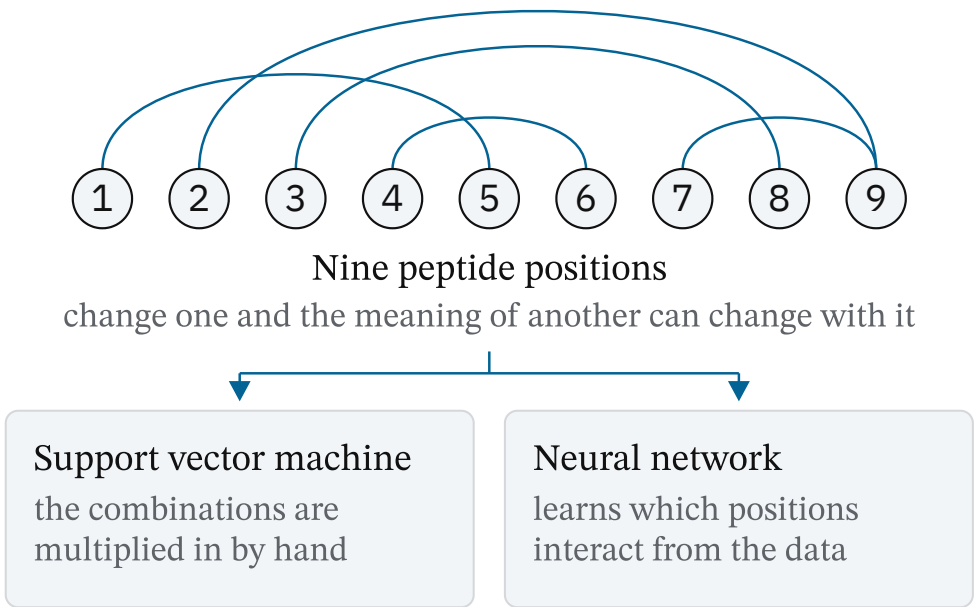


Figure 6.1 Why the model changes in this chapter. The nine peptide positions do not act one at a time: change one and the meaning of another can change with it. A support vector machine can be given those combinations by hand, while a network learns which of them matter from the data.

6.2 THE DATA

Most of what is known about T-cell response is not about cancer at all.

The data this chapter needs comes from IEDB, the same database Chapter 2 used. It records whether a peptide, once shown to a T cell, actually provoked a response. That is immunogenicity data, and it is rarer than binding data. Most peptides in IEDB have

only ever been tested for binding, not for response. Of the peptides that do have a response recorded, most come from viruses and other pathogens. Tumour-derived neoantigens are a small fraction of the whole.

There is a second problem underneath that one. A peptide that never binds MHC in the first place cannot be immunogenic, because it is never shown to a T cell at all. If a model trains on all peptides, bound and unbound together, it can learn to predict binding and call that immunogenicity. The two get confounded.

So the rule going in is this. Every peptide used here, for training or for testing, already binds MHC. The question the model answers is narrower than “is this peptide dangerous”. It is “given a peptide already sitting on the cell surface, does a T cell actually notice it”.

That narrows the training data further, to mostly pathogen epitopes that bind and provoke a response, tested against the much smaller set of tumour neoantigens that do the same. TESLA, a benchmark of real tumour neoantigens with confirmed T-cell response, is the largest of those. The model trains on the broad set. It gets tested, once, on the narrow one (Figure 6.2).

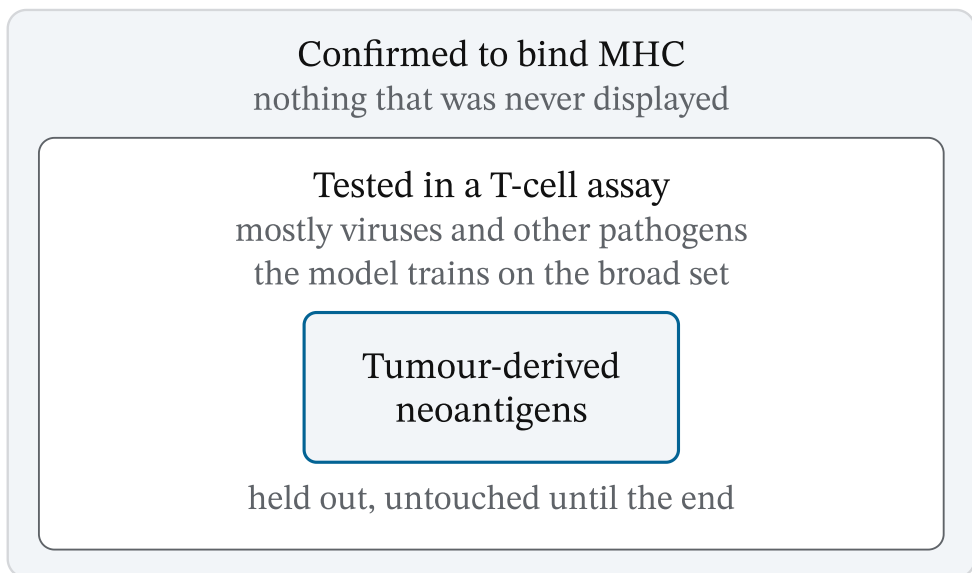


Figure 6.2 The nested problem. Every peptide used here already binds MHC, which is the outer box; inside it are the peptides that have been tested in a T-cell assay, positive or negative, which come mostly from viruses and other pathogens and are the set the model trains on; inside that again are the tumour-derived neoantigens, a small fraction of the whole, held out for the final test.

6.3 THE RULES

These rules were written before any pipeline ran.

RULES FOR THIS CHAPTER

Data.

IEDB's T-cell assay records, restricted to HLA-A*02:01, the same allele Chapter 2 used. Every peptide included already has confirmed MHC binding, nothing that was never displayed. TESLA is held out completely, untouched until the very end.

Features.

The nine peptide positions, nothing else. No HLA sequence, no clinical data, no patient information. HLA-A*02:01 is fixed across the whole dataset, so it cannot be a feature.

Outcome.

Positive means a T-cell response was recorded in the assay. Negative means the peptide was tested and no response was detected. That definition is fixed before anyone looks at how many of each there are.

Split.

Peptides are clustered by sequence similarity first, then split by cluster, not by individual peptide. Near-identical peptides landing on both sides of the split would let the model memorise instead of learn.

Metric.

Immunogenic peptides are the minority class here, so accuracy is not the number that matters. The primary metric is the area under the precision-recall curve (AUPRC), reported alongside standard AUC. Chance is not 0.5 this time. Chance is whatever fraction of the dataset is actually positive.

Baseline.

A model that predicts immunogenicity using only the binding score, nothing about the sequence itself. If the network cannot beat a model that only knows binding strength, sequence pattern recognition added nothing.

6.4 FOUR WAYS THIS COULD BE WRONG

I wrote these down in advance as well (Figure 6.3).

One. Assay negative is not biological negative. A T-cell assay tests one donor's blood at one point in time. A peptide marked non-immunogenic may simply have met a donor whose repertoire did not include the right T cell that day. Absence of response in the data is not proof that the peptide can never provoke one.

Two. The training data still leans pathogen, not tumour, even after restricting to confirmed binders. Most immunogenic peptides in IEDB come from viruses. A pattern the network learns there may not transfer cleanly to a mutated self protein on a tumour cell.

Three. HLA-A*02:01 only. A large share of people do not carry this allele at all. Whatever this model learns says nothing about immunogenicity on any other HLA type, and that scope stays in the headline, not the fine print.

Four. Memorisation. The positive class here is small, and small positive classes are exactly where a network can learn the specific peptides in front of it rather than the pattern behind them. Clustering by similarity at the split reduces this. It does not remove it, and performance on TESLA is the real check.

- 1 Assay negative is not biological negative
one donor, one point in time
- 2 The training data leans pathogen
most immunogenic peptides in IEDB come from viruses
- 3 One allele only, HLA-A*02:01
nothing is learned about any other HLA type
- 4 Memorisation
learning the specific peptides, not the pattern

Figure 6.3 The four ways the chapter could be wrong, written down in advance: a negative assay result is not a biological negative, the training data leans towards pathogens, only one HLA allele is covered, and the model may learn the particular peptides rather than the pattern behind them.

6.5 WHAT HAPPENED

I built this chapter's model to clearly beat a simpler one. It did not.

The cohort came to 1,200 peptides, all confirmed to bind HLA-A*02:01. Of those, 606 were immunogenic and 594 were not. The rules had been written expecting the immunogenic peptides to be the minority class, and the memorisation risk rested on the same expectation. At 50.5% of the cohort they were a slim majority instead.

The label also hides a choice. A peptide counts as immunogenic if any one of its assays recorded a response, and 340 of the 1,200, 28.3%, have at least one assay that did and at least one that did not. All 340 are labelled immunogenic, and every number in this chapter inherits that ambiguity.

I split by cluster rather than by peptide. Near-identical variants of the same epitope are not the rule in this data: 1,061 of the 1,098 clusters hold a single peptide, and only 139 of the 1,200 peptides sit in a cluster with any other. That small group is what a random split would have spread across both sides, leaving 13.7% of validation peptides within two substitutions of a training peptide. The cluster split brought that to zero, and left 899 peptides for training and 301 for validation, 146 of them immunogenic. Listing 6.1 shows how the clusters are formed.

Listing 6.1 Joining every pair of peptides that differ at no more than two positions and taking each connected group as one cluster, from `scripts/02_cluster_split.py`.

```
n = len(peptides)
codes = np.array([[common.AA_INDEX[a] for a in p] for p in peptides],
dtype=np.int8)

parent = np.arange(n)

def find(x: int) -> int:
    while parent[x] != x:
        parent[x] = parent[parent[x]]
        x = parent[x]
    return x

def union(a: int, b: int) -> None:
    ra, rb = find(a), find(b)
    if ra != rb:
        parent[max(ra, rb)] = min(ra, rb)

for i in range(n - 1):
    # distance from peptide i to every peptide after it, in one vector op
    d = (codes[i + 1:] != codes[i]).sum(axis=1)
    for j in np.nonzero(d <= max_distance)[0]:
        union(i, i + 1 + int(j))

roots = np.array([find(i) for i in range(n)])
_, cluster_id = np.unique(roots, return_inverse=True)
return cluster_id
```

Area under the precision-recall curve (AUPRC). Precision is the share of the peptides a model calls immunogenic that really are. Recall is the share of the immunogenic peptides it finds. The area under the curve of one plotted against the other is a single number for how well a model ranks the positive class, taken across every threshold at once rather than at a single cut-off. A model that ranks at random scores the positive fraction of whatever set it is scored on, which is why chance moves with the class balance instead of sitting at 0.5. The standard AUC reported beside it is the area under the ROC curve.

I compared the two architectures, the feedforward network and the interaction model. The second was expected to win. It did not win clearly. All six configurations I tried scored between 0.603 and 0.615 AUPRC on the outer validation peptides, inside each other's seed-to-seed noise, the spread a single configuration shows when it is run again from a different random starting point. The rule fixed in advance selected the feedforward network, using an inner validation split taken from inside the training data. On the outer validation split the ranking flipped, and the interaction model scored higher. Both numbers belong here, not only the one that favours the winner.

The core comparison came out just as narrow. Every peptide here is a confirmed binder, but confirmed binders still differ in how tightly they bind, so an affinity score has something left to rank them by. On the outer validation peptides the baseline using only binding affinity, with no sequence at all, scored 0.591 AUPRC. The selected network scored 0.610. Chance there is 0.485, the fraction of the 301 validation peptides that are immunogenic, and not the 0.505 of the cohort as a whole. That margin of 0.019 is the same size as the network's own run-to-run variation.

The standard AUC the rules promised beside the AUPRC puts the two in the same order. The selected network reached 0.665 on the validation peptides, and the binding-affinity baseline 0.629, against 0.5 for chance, which on that curve does not move with the class balance.

6.6 WHAT THE RESULT DOES AND DOES NOT SHOW

One result ran backwards from expectation. Weaker binders were more often immunogenic than tighter ones. Two explanations fit equally well. The immune system may delete the T-cell clones that would recognise tight-binding, self-like peptides; or tightly measured peptides may simply be the ones researchers chose to test. This data cannot tell which.

The bigger gap is the pre-registered final test, on TESLA's tumour-derived peptides, which never ran. I could not get the table of validated peptides. The public Synapse deposit, syn21048999, holds only raw whole-exome and RNA sequencing, with no

peptide-level table in it. The table is published in the supplementary material of the TESLA paper, and at PMC that download was blocked by a browser challenge, while Europe PMC would not serve the article either, reporting it as not open access. A person can still fetch that file in a browser, and the script that would score it is written and has been exercised on a synthetic table, so what is missing is the file itself. The headline number here has not been checked against real tumour neoantigens. It is the same pattern as Chapter 5. The real result is sometimes about what data you can actually reach, rather than the biology you set out to test.

Six chapters have now taken up six models, the fifth of them planned and never run, and every model that did run was handed the correct label during training and asked to reproduce it, the last of them a network learning for itself which combinations of positions mattered. Chapter 7 has no answer key. The model sees only the data and has to find structure inside it unaided, the way a biologist already does at a bench, sorting cells by what they look like rather than by a name someone assigned in advance. One chapter is left, and the book ends there on purpose, at a real stopping point, not because the models ran out.

This chapter is rewritten from the posts for Days 55 to 61 of the series.

RUN IT

Repository github.com/Qasim-Hussain-Code/neoantigen_immunogenicity_prediction
Commit [68a12d14824d](#)

CITE THIS CHAPTER

Hussain, Q. (2026). Neoantigen immunogenicity for cancer vaccine design. In *Machine Learning for Biology*, chapter 6.

PART II

LEARNING WITHOUT LABELS

IMMUNE CELL SUBTYPE DISCOVERY FROM SINGLE-CELL RNA-SEQ

Model: k-means

Given only a cell's gene expression, can an algorithm that has never seen a cell type label recover the immune cell populations a lab already knows are there?

SUMMARY

Blood cells from ten bead-enriched populations, 94,655 in all, were pooled with their labels locked away, and k-means was asked to group them from gene expression alone. Left to choose the number of groups by the silhouette, it chose 2, and the run fixed in advance lost to a baseline given only three numbers about how well each cell was measured, scoring 0.01 against 0.04 on the adjusted Rand index for the ten labels, although a different two-group split, found by 12 of 20 reruns from other starting points, scored 0.06 in a comparison not fixed in advance. Told to make ten groups, it scored 0.44 on the ten labels and 0.55 on six broad lineages, against 0.08 and 0.09 for the baseline. Four of six predictions written down in advance passed, and the helper T-cell tube did not separate from the naive T cells nested inside it, which argues against the groups being driven by the tubes.

Draw a tube of blood, spin it down, and the thin white layer in the middle holds T cells, B cells, NK cells, monocytes and dendritic cells, tangled together. Under a microscope, several of them look nearly identical.

Every chapter before this one told the model the correct answer during training. Logistic regression knew which genomes carried resistance. The network in Chapter 6 knew which peptides provoked a response. The first chapter raised a different possibility and then left it alone for six chapters: biologists already sort cells into types without any of that supervision. This chapter is where that gets tested properly.

The question is this. Given only a cell's gene expression, nothing else, can an algorithm that has never seen a cell type label recover the immune cell populations a lab already knows are there?

The check is what makes the question answerable rather than just interesting. The cells used here were separated into ten populations by an entirely different method from transcriptomics. Those labels never touch the clustering. They exist only to be compared against afterwards (Figure 7.1).

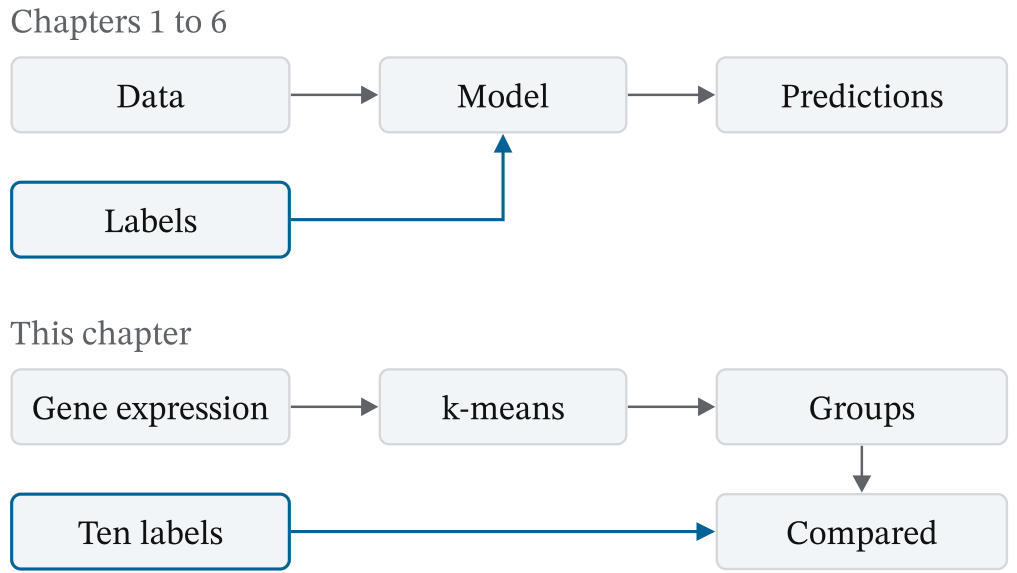


Figure 7.1 Where the labels go. In Chapters 1 to 6, the labels went into training the model; in this chapter, k-means sees only gene expression, and the ten labels meet its groups only afterwards, when the two are compared.

One limitation needs naming at the outset. Clustering finds structure, not names. It can group T cells together without knowing to call them T cells, and it can just as easily split one real population into two or merge two distinct ones, especially where the biology itself is gradual rather than discrete. Whatever comes out still needs a human, and marker genes, the genes that set one cell type apart from the rest, to say what each group actually is.

7.1 WHAT THE LABELS CAN AND CANNOT SEE

The ten populations were pulled out of one donor’s blood with antibody-coated beads, and flow cytometry came in only afterwards, to check how pure each tube was. So every label here, the whole answer key, was made with an antibody against a protein on the cell surface: CD4, CD8, CD14, CD19, CD25, CD34, CD45RA, CD45RO and CD56.

The algorithm will never see a protein. It sees RNA, counted from the tail end of each message. Proteins and messages usually agree. In these cells there are three places where they do not.

One label is close to invisible. Naive and memory T cells were separated using CD45RA versus CD45RO. Those are not two genes. They are two versions of one gene, *PTPRC*, made by cutting different pieces out of the same message, and those pieces sit near the front, far from the tail this chemistry reads. A 2024 study tested exactly this on tail-end data from blood cells: even with very deep sequencing, fewer than one cell in ten showed either version.

Some labels overlap by definition. The helper T cells were selected on CD4 alone. Naive, memory and regulatory T cells all carry CD4, so the helper tube holds cells that match three other labels. The two CD8 populations, cytotoxic and naive, nest in the same way, with a twist: some of the most experienced killer T cells switch CD45RA back on, so “naive” here is not purely naive.

One label is mostly something else. By the dataset’s own flow check, the CD34+ tube was 45% pure.

In Chapter 5, the gene did not follow the organism. Here the label does not fully follow the RNA.

One decision was made before any code ran, rather than after a disappointing score. The results would be scored against two answer keys. The first is the ten labels, one per tube: B cells, monocytes, NK cells, CD34+ cells and six T-cell labels, four of CD4 T cells (helper, naive, memory and regulatory) and two of CD8 T cells (cytotoxic and naive). The second is six broad lineages: B cells, monocytes, NK cells, CD34+ progenitors, CD4 T cells and CD8 T cells, so that the four CD4 T-cell tubes become one family. Both were declared in advance, so neither could be chosen later to flatter the result.

7.2 WHAT K-MEANS DOES

Each cell starts as counts for thousands of genes. Many of those genes rise and fall together, so they can be squeezed into 50 combined axes that keep the main patterns. That squeeze is called principal component analysis. Now picture every cell as a point in a space with those 50 directions. Similar cells sit close together.

k-means drops k markers into that space. Every cell joins its nearest marker. Each marker then moves to the middle of the cells that joined it. Cells switch to whichever marker is now nearest, and the process repeats until nothing moves.

That is the whole algorithm. There is no answer key anywhere. The one instruction it gets is k , the number of groups (Figure 7.2).

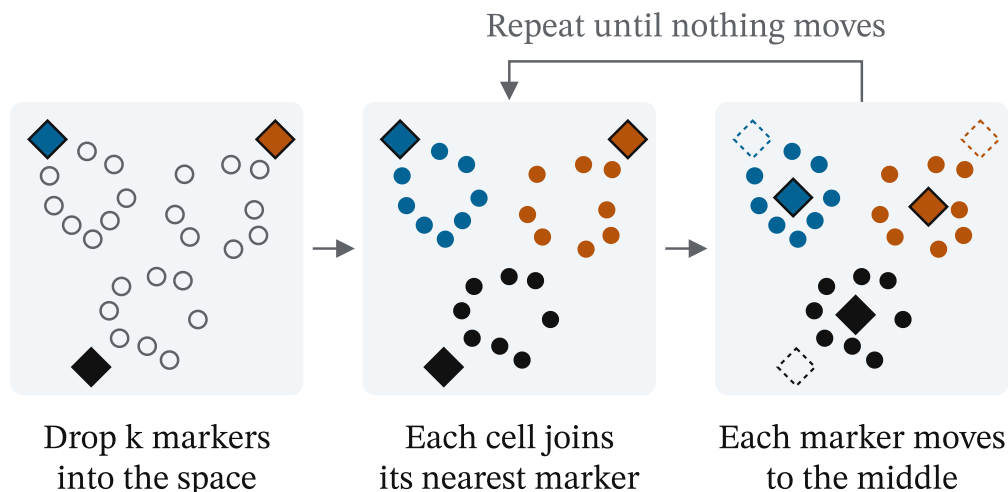


Figure 7.2 One round of k-means with three markers, drawn on a sketch rather than the chapter's data; each point stands for a cell. Markers are dropped into the space, each cell joins its nearest marker, and each marker moves to the middle of the cells that joined it; k , the number of groups, is the one instruction the method is given.

Chapter 5 set out to use a support vector machine, a model that searches for the widest gap between two outcomes it has been told about. k-means has no outcomes. It decides where the gaps are on its own.

It is not a toy choice for this data. The 2017 study that produced these cells clustered with k-means, on 50 principal components, and for its 68,000-cell blood sample it set k to 10 after reading an error curve. Reading a curve like that is a judgement call made after seeing the data. This chapter replaces it with a rule fixed in advance.

There are two reasons k-means closes the book. It is the plainest version of the idea, the unsupervised counterpart to Chapter 1's logistic regression. And it drags the most important decision into the open: how many groups are there? Most methods answer that quietly, inside a setting. k-means will not start until someone answers it.

One weakness shows up at the start line. Where the markers begin can change where they finish. So every run starts from 50 different random positions and keeps the tightest result.

k-means is not what single-cell labs usually reach for. Seurat and Scanpy, two of the most widely used toolkits, default to graph-based clustering: link each cell to its nearest neighbours, then find communities more tightly linked inside than out. The version used here is called Leiden. It needs no k, only a resolution dial that does the same job less visibly. Leiden ran alongside k-means, its dial set by the same rule that picks k. If the textbook method lost to the field's default, that would be reported too.

7.3 THE RULES

The rules were fixed before any code ran.

RULES FOR THIS CHAPTER

Data.

The ten bead-enriched populations released by 10x Genomics alongside Zheng and colleagues, *Nature Communications*, 2017. Every cell the original pipeline called a cell, pooled into one table. Nothing removed on top by thresholds of my choosing.

Answer key.

It goes in a locked drawer. At the very first step the labels move to a separate file, every cell gets a random code in place of its name, and the order is shuffled. No script that groups cells may open that file. It opens once, after the groups are final and committed to the repository.

Input.

Gene expression and nothing else. The settings are borrowed, not tuned: the processing recipe from the 2017 paper, which the Scanpy library ships under that paper's name. The 1,000 most variable genes, squeezed to 50 axes. No correction for sequencing batch, because each tube was measured separately: here batch and label are the same thing, and correcting for one would erase the other.

Number of groups.

Anything from 2 to 20 is allowed. The winner is the number where cells sit most clearly inside their own group rather than the next one over, a measure called the silhouette, and the same rule sets Leiden's dial. The number is never set to ten just because the key says ten. One extra run, labelled as borrowing from the key, forces ten groups, so that a wrong count can be told apart from a wrong grouping.

Metric.

The adjusted Rand index, against the ten labels and against the six lineages, with a 95% confidence interval. Its zero point is checked by shuffling the key 1,000 times.

Baseline.

The same model, fed three numbers per cell that describe the measurement rather than the cell: molecules captured, genes detected and the share of molecules from mitochondrial genes. Gene expression beats it only if the whole 95% interval for its lead sits above zero. If gene expression cannot clearly beat it, the groups describe the machine.

Names before answers.

Before the key opens, each group of the k-means run that chooses its own number gets a name from its marker genes, twice: once by a fixed rule written down in advance, once by me. Both are committed, then scored.

Everything on record.

Every setting, down to the random seed, goes into the repository before the analysis starts, along with a pass-or-fail test for each prediction.

Adjusted Rand index. A measure of how far two ways of grouping the same cells agree, judged by whether each pair of cells is kept together or kept apart in both. The groups need no names, and the two groupings need not have the same number of groups. 0 is what random grouping scores on average, and 1 is perfect agreement.

The locked drawer is a few lines of code in the script that builds the pooled table (Listing 7.1). Figure 7.3 sets out the protocol as a whole.

Listing 7.1 Shuffling, renaming and setting the labels aside, from `scripts/02_build_matrix.py`.

```
counts = sp.vstack(blocks, format="csr")
order = rng(cfg["rng_streams"]["shuffle"]).permutation(counts.shape[0])
counts = counts[order]
keys = np.asarray(keys)[order]
barcodes = np.asarray(barcodes)[order]
ids = [f"c{i:06d}" for i in range(1, counts.shape[0] + 1)]

obs = pd.DataFrame(index=pd.Index(ids))
var_out = pd.DataFrame({"gene_symbols": var["gene_symbols"].to_numpy()},
index=pd.Index(var.index.to_numpy()))
pooled = ad.AnnData(X=counts, obs=obs, var=var_out)
pooled.uns.clear()
for column in list(pooled.obs.columns):
    del pooled.obs[column]
pooled_path = ROOT / qcfg["pooled_file"]
pooled_path.parent.mkdir(parents=True, exist_ok=True)
pooled.write_h5ad(pooled_path, compression="gzip")

labels_path = ROOT / qcfg["labels_file"]
pd.DataFrame({"cell_id": ids, "population_key": keys, "original_barcode":
barcodes}).to_csv(labels_path, index=False, lineterminator="\n")
```

- Settings and pass-or-fail tests committed

Answer key locked away

- Labels moved to a separate file; cells recoded and shuffled
 - Gene expression only, squeezed to 50 axes
 - The silhouette picks k , from 2 to 20
 - Groups named from marker genes and committed
-
- The key opens once, and the groups are scored

Figure 7.3 The protocol in order. The settings and the pass-or-fail tests are committed first; the next four steps, from moving the labels to a separate file to naming the groups, run while the answer key is locked away; and the key opens once, at the end, to score the groups.

7.4 PREDICTIONS, AND HOW THIS COULD GO WRONG

I wrote down predictions before any code ran. They were not blind: others had clustered these cells before. B cells and monocytes would come back largely intact. NK cells would partly blur into cytotoxic T cells, because both run the same killing programme. The six T-cell labels would not separate cleanly, and the helper label could not separate from the populations nested inside it. The CD34+ label would mostly hold together, as it did in a 2020 re-analysis. That is odd for a tube that is 45% pure, and odd results get checked against the sequencing batch first.

I also set out the ways the chapter could go wrong, each with a check.

The first is that it might sort cells by the tube they came in. Imagine ten piles of holiday photos, each taken with a different camera. Ask someone to sort them by who is in them, and they might sort them by camera instead, because one camera's photos are brighter and another's are blurrier. The same risk exists here. Each cell type arrived in its own tube and was measured separately, some far more thoroughly than others. The check is the baseline in the rules. Its three numbers are the "camera settings": they say how well each cell was measured and nothing about the cell itself. The real data has to clearly beat them. If it cannot, the groups are about the camera, not the people. A second check uses the helper T cells. That tube was collected with a broad net, so it

already holds many cells from another tube, the naive T cells. Those two should be hard to separate. If the computer separates them more easily than two tubes that really are different, it is sorting by tube.

The second is that it might count wrong but sort right. The computer decides how many groups there are by looking for the cleanest splits. In blood, the cleanest splits are the big ones, so it may find three or four groups where the lab defined ten. The check is to publish its score for every group count from 2 to 20, plus one extra run told to make ten.

The third is that it never says “I am not sure”. Every cell must go into some group, even cells caught halfway through changing from one type to another. And where the computer starts is random, which can change where it ends. The check is to run it 20 times from different starting points and to compare it with Leiden.

The fourth is that a strange group is not a discovery. A group that matches no label might be a new cell type. It is just as likely to be stray cells, two cells stuck together, or dying cells. Any such group is called “unresolved” and its genes are listed. No new cell types will be claimed.

7.5 BEFORE THE KEY OPENED

Everything in this section was frozen and committed to the repository before the answer key opened.

94,655 blood cells went in. Each one was just a list of gene counts, with no names attached. The processing recipe, asked for the 1,000 most variable genes, returned 999, and I used its output as it was. The first job was to decide how many groups to make. The rule tried every number from 2 to 20 and kept the one where cells sat most clearly inside their own group (Listing 7.2).

Listing 7.2 The rule that chooses the number of groups, from `scripts/04_select_k.py`.

```
def pick(values, scores):
    """Highest score after rounding to 4 decimal places; ties go to the smallest
    k or lowest resolution."""
    rounded = {v: round(s, 4) for v, s in zip(values, scores) if s is not None}
    best = max(rounded.values())
    return min(v for v, s in rounded.items() if s == best)
```

It picked 2. Not ten, and not six. One group of 3,521 cells full of monocyte genes, and one group of 91,134 cells holding everything else. This is what the second risk warned about: in blood, the cleanest split is a coarse one. The silhouette for two groups was

0.63. For ten groups it was 0.21.

So the extra run, the one told to make exactly ten groups, came to matter a great deal. On their marker genes, its groups look like real cell families: B cells, NK cells, monocytes, several kinds of T cells and three separate groups the naming rule called CD34+ progenitors. Leiden, the kind of method most labs use, settled on 23 groups.

Then came the 20 reruns from different random starting points. Leiden gave much the same answer each time. k-means did not. At two groups, 12 of the 20 reruns found a different split from the run fixed in advance, and a slightly tighter one, with 83,404 cells in one group and 11,251 in the other. The rules say the pre-committed run is the one that gets scored, so it is. The other answer is reported next to it.

Finally, the two groups k-means chose for itself were named from their genes, twice. The fixed rule called the big group “CD8 T cells”. The second naming, the one I had said I would do myself, worked from the same marker genes before the key opened, and called the big group “unresolved”: a mix of several cell types. The fixed rule also named every group of the ten-group run and of Leiden.

7.6 AGAINST THE ANSWER KEY

Every grouping beat random easily. I shuffled the labels 1,000 times, and the highest any shuffle scored was 0.003.

That was the warm-up. The real test was the camera-settings baseline: the same method, k-means, given only the three numbers about how well each cell was measured.

At the two groups k-means chose for itself, gene expression lost. On the ten labels it scored 0.01 and the camera settings 0.04. On the six lineages it scored 0.05 and the camera settings 0.13. The reason is almost boring. Gene expression split off a small group, mostly monocytes. The camera settings split off the cells that gave the most RNA. The CD34+ tube gave far more RNA per cell than any other, so 89% of it landed in that group. Against the labels, the camera-settings split happens to score better.

The other two-group answer did better. The split that 12 of the 20 reruns found scored 0.06 on the ten labels and 0.18 on the six lineages, above the camera settings’ 0.04 and 0.13. That comparison was not part of the plan and has no interval, so the verdict still rests on the run fixed in advance.

Then I told k-means to make ten groups, the same number as the lab's labels. This time gene expression won clearly: 0.44 against 0.08 on the ten labels, and 0.55 against 0.09 on the six lineages.

Leiden, with its 23 groups chosen by the same rule, beat two-group k-means on both keys, with 0.58 on the ten labels and 0.42 on the six lineages. Against ten-group k-means it was a split decision. Leiden scored higher on the ten labels, 0.58 against 0.44, and k-means scored higher on the six lineages, 0.55 against 0.42. That comparison was not planned in advance, so it stays a description rather than a verdict.

The result fits in two sentences. When k-means has to decide how many groups exist, it gets the count wrong, and the camera settings beat it. When it is told the count, it recovers a large part of the lab's structure from RNA alone. The second risk in my list was "count wrong but sort right", and that is what happened.

The names were scored too. A name counted as correct if it matched the group's most common lineage, where that lineage made up at least half of the group, or if it said "unresolved" where no lineage did. The big group k-means chose for itself was only 46% CD4 T cells, its most common lineage, so "unresolved" was the right name and "CD8 T cells" was wrong. The fixed rule got 1 of those 2 groups right, and my second naming got both. My naming covered only those two groups, so its perfect score is a small test. On the ten-group run, the fixed rule named 9 of 10 groups correctly, and on Leiden, 17 of 23. Every group it missed on those two runs was a small one.

7.7 THE SIX PREDICTIONS

The predictions were scored on the run told to make ten groups, because they are about ten labels. Four passed. Two failed (Figure 7.4).

Overlap and pair score. The overlap of two labels adds up, over every group, the smaller of two shares: the share of the first label's cells that sit in that group, and the share of the second label's cells that sit there. It is 0 when the two labels never share a group, and 1 when they are spread across the groups in exactly the same way. The pair score is the adjusted Rand index computed on only the cells that carry one of the two labels, so it measures how cleanly the groups separate those two labels.

B cells came back intact, a pass. 99.9% of them landed in one group, and that group was 96% B cells. Monocytes also passed: 93% of them landed in one group.

The NK-cell prediction failed. It needed an overlap with cytotoxic T cells of at least 0.10, and the overlap came out at 0.05. Sharing a killing programme was not enough to mix them: 98% of NK cells sat in one group that was 96% NK cells.

The CD34+ prediction also failed, and it is the most interesting miss. The tube did not scatter into other cell types. Most of it split into three groups of its own, each more than 98% cells from the CD34+ tube. But its biggest single group held only 30% of it, far short of the 80% needed. This was the one prediction shaped by the 2020 re-analysis, where most of these cells stayed in one cluster.

The predictions said odd results get checked against the sequencing batch first, and this result is odd in that way: a tube that was 45% pure kept most of its cells in groups of its own. In the biggest of those groups, 34% of cells had at least one count of the *CD34* gene, against 2% of all the cells outside it, so that group does carry the *CD34* message. Among cells from the CD34+ tube, the share was 34% inside that group and 32% outside it. By this measure, the tube's cells outside its biggest group resemble the ones inside it rather than contaminants. What the check cannot do is separate the cells from their run. The camera settings alone kept much of the tube together: 89% of it in one group at two groups, and 54% at ten. Batch and label are the same thing here, so nothing in this chapter can tell how far the tube stands apart because of its cells and how far because of its run.

The six T-cell labels did not separate cleanly, a pass. Not one of them got a group holding 80% of its cells at 80% purity.

The last prediction is the one that matters most: the helper tube could not be pulled apart from the naive T cells inside it. It passed. Helper against naive separated with a pair score of 0.05. Naive against memory, two genuinely different kinds of T cell, separated at 0.49. If the tubes, rather than the cells, had been driving the groups, that order would have flipped. It did not.

Four passed

B cells come back largely intact

99.9% in one group, which was 96% B cells

Monocytes come back largely intact

93% in one group

The six T-cell labels do not separate cleanly

None got a group with 80% of its cells at 80% purity

The helper tube does not separate from naive T cells

Pair score 0.05, against 0.49 for naive and memory

Two failed

NK cells partly blur into cytotoxic T cells

Overlap 0.05, needed at least 0.10

The CD34+ tube mostly holds together

Its biggest group held 30% of it, needed 80%

Figure 7.4 The six predictions, scored on the run told to make ten groups, each with the number that decided it. The low overlap means NK and cytotoxic T cells stayed apart, so that prediction failed; the equally low pair score means the helper and naive tubes did not separate, so that one passed.

Because others had clustered these cells before, B cells and monocytes were safe calls. The two misses are the reason predictions get written down first. Without a record made before any code ran, it would be easy to claim afterwards that these results were expected.

This chapter is rewritten from the posts for Days 62 to 69 of the series.

REFERENCES

Zheng, G. X. Y., Terry, J. M., Belgrader, P. et al. (2017). Massively parallel digital transcriptional profiling of single cells. *Nature Communications*, 8, 14049. <https://doi.org/10.1038/ncomms14049>

RUN IT

Repository github.com/Qasim-Hussain-Code/immune_cell_subtype_discovery

Commit [1141fa411573](https://github.com/Qasim-Hussain-Code/immune_cell_subtype_discovery/commit/1141fa411573)

CITE THIS CHAPTER

Hussain, Q. (2026). Immune cell subtype discovery from single-cell RNA-seq. In *Machine Learning for Biology*, chapter 7.

EPILOGUE

Seven chapters, seven models, and the basics of machine learning, one idea at a time.

Chapter 1 used logistic regression: a straight line drawn through a decision. Chapter 2 used a decision tree, a model you can read out loud, one split at a time. In Chapter 3, many trees voted together as a random forest, and in Chapter 4, gradient boosting added each new tree to fix what the last one got wrong. Chapter 5 set out to use a support vector machine, a model that looks for the widest possible gap between two outcomes, but the public data could not support the design it was chosen for. It answered two narrower questions with logistic regression instead. Chapter 6 used a neural network, a model that learns on its own which combinations of positions in a peptide matter. Chapter 7, on immune cells profiled by single-cell RNA-seq, used k-means: no answer key at all, only the data.

The first six were supervised. They learned from examples someone had already labelled. The last was unsupervised. It had to find the groups by itself, and it showed exactly where that gets hard. Sorting the cells, once it was told how many groups to make, was the easy part. Deciding how many groups there are was not.

The models changed from chapter to chapter. The approach did not. Fix the rules before fitting anything. Compare against a simple baseline. Share what did not work as openly as what did.

That approach is better than the model. A result nobody can check is only an opinion.

This epilogue is rewritten from the post for Day 70 of the series.